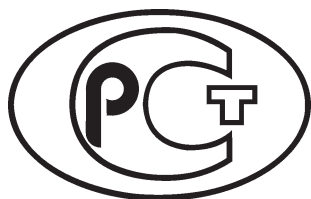

ФЕДЕРАЛЬНОЕ АГЕНТСТВО
ПО ТЕХНИЧЕСКОМУ РЕГУЛИРОВАНИЮ И МЕТРОЛОГИИ



РЕКОМЕНДАЦИИ
ПО СТАНДАРТИЗАЦИИ

Р 1323565.1.035—
2021

Информационная технология

КРИПТОГРАФИЧЕСКАЯ ЗАЩИТА ИНФОРМАЦИИ

Использование российских криптографических
алгоритмов в протоколе защиты информации ESP

Издание официальное



Москва
Стандартинформ
2021

Предисловие

1 РАЗРАБОТАНЫ Акционерным обществом «ЭЛВИС-ПЛЮС» (АО «ЭЛВИС-ПЛЮС») при участии специалистов Общества с ограниченной ответственностью «Фактор-ТС» (ООО «Фактор-ТС»), Общества с ограниченной ответственностью «КРИПТО-ПРО» (ООО «КРИПТО-ПРО») и Общества с ограниченной ответственностью «С-Терра СиЭсПи» (ООО «С-Терра СиЭсПи»)

2 ВНЕСЕНЫ Техническим комитетом по стандартизации ТК 026 «Криптографическая защита информации»

3 УТВЕРЖДЕНЫ И ВВЕДЕНЫ В ДЕЙСТВИЕ Приказом Федерального агентства по техническому регулированию и метрологии от 28 января 2021 г. № 27-ст

4 ВВЕДЕНЫ ВПЕРВЫЕ

Правила применения настоящих рекомендаций установлены в статье 26 Федерального закона от 29 июня 2015 г. № 162-ФЗ «О стандартизации в Российской Федерации». Информация об изменениях к настоящим рекомендациям публикуется в ежегодном (по состоянию на 1 января текущего года) информационном указателе «Национальные стандарты», а официальный текст изменений и поправок — в ежемесячном информационном указателе «Национальные стандарты». В случае пересмотра (замены) или отмены настоящих рекомендаций соответствующее уведомление будет опубликовано в ближайшем выпуске ежемесячного информационного указателя «Национальные стандарты». Соответствующая информация, уведомление и тексты размещаются также в информационной системе общего пользования — на официальном сайте Федерального агентства по техническому регулированию и метрологии в сети Интернет (www.gost.ru)

© Стандартиформ, оформление, 2021

Настоящие рекомендации не могут быть полностью или частично воспроизведены, тиражированы и распространены в качестве официального издания без разрешения Федерального агентства по техническому регулированию и метрологии

Содержание

1 Область применения	1
2 Нормативные ссылки.....	1
3 Термины, определения, обозначения и сокращения.....	2
3.1 Термины и определения	2
3.2 Обозначения	2
3.3 Сокращения	2
4 Краткое описание протокола ESP	3
4.1 Формат ESP-пакета	3
4.2 Режимы протокола ESP.....	7
4.3 Дополнительное инкапсулирование ESP-пакетов	8
5 Обработка пакетов	10
5.1 Политика безопасности.....	10
5.2 Защищенное соединение (SA)	10
5.3 Обработка исходящего пакета.....	10
5.4 Обработка входящего пакета	15
6 Использование российских криптографических алгоритмов в протоколе ESP.....	16
6.1 Согласование параметров ESP SA	16
6.2 Трансформы.....	16
6.3 Идентификаторы алгоритмов.....	21
Приложение А (справочное) Максимальный объем обрабатываемого трафика	22
Приложение Б (справочное) Тестовые векторы.....	23
Приложение В (справочное) Алгоритм обработки расширенного номера пакета.....	28
Библиография	29

Введение

Настоящие рекомендации содержат описание протокола защиты информации ESP (Encapsulating Security Payload) при использовании в нем российских криптографических алгоритмов, определенных в ГОСТ Р 34.12—2015, Р 50.1.113—2016 и Р 1323565.1.026—2019.

Необходимость создания настоящего документа вызвана потребностью в обеспечении совместимости реализаций протокола ESP от различных производителей при использовании российских криптографических алгоритмов.

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ТЕХНИЧЕСКОМУ РЕГУЛИРОВАНИЮ И МЕТРОЛОГИИ

РЕКОМЕНДАЦИИ ПО СТАНДАРТИЗАЦИИ

Информационная технология

КРИПТОГРАФИЧЕСКАЯ ЗАЩИТА ИНФОРМАЦИИ

Использование российских криптографических алгоритмов
в протоколе защиты информации ESPInformation technology. Cryptographic data security.
Using Russian cryptographic algorithms in data security protocol ESP

Дата введения — 2021—06—01

1 Область применения

Протокол ESP (Encapsulating Security Payload) входит в семейство протоколов IPsec и предназначен для защиты информации в сетях TCP/IP. Данные рекомендации описывают применение ESP при использовании в нем криптографических алгоритмов, определяемых документами национальной системы стандартизации Российской Федерации. Протокол позволяет обеспечить такие свойства безопасности, как конфиденциальность и имитозащита передаваемых данных, а также защита от атак повторной передачи пакетов. Протокол может применяться при защите информации, не содержащей сведений, составляющих государственную тайну, в частности, для построения виртуальных частных сетей (Virtual Private Network).

Настоящие рекомендации базируются на [1] и расширяют их в плане применения российских криптографических алгоритмов.

2 Нормативные ссылки

В настоящих рекомендациях использованы нормативные ссылки на следующие документы:

ГОСТ Р 34.12—2015 Информационная технология. Криптографическая защита информации.

Блочные шифры

Р 50.1.113—2016 Информационная технология. Криптографическая защита информации. Криптографические алгоритмы, сопутствующие применению алгоритмов электронной цифровой подписи и функции хэширования

Р 1323565.1.026—2019 Информационная технология. Криптографическая защита информации. Режимы работы блочных шифров, реализующие аутентифицированное шифрование

П р и м е ч а н и е — При пользовании настоящими рекомендациями целесообразно проверить действие ссылочных документов в информационной системе общего пользования — на официальном сайте Федерального агентства по техническому регулированию и метрологии в сети Интернет или по ежегодному информационному указателю «Национальные стандарты», который опубликован по состоянию на 1 января текущего года, и по выпускам ежемесячного информационного указателя «Национальные стандарты» за текущий год. Если заменен ссылочный документ, на который дана недатированная ссылка, то рекомендуется использовать действующую версию этого документа с учетом всех внесенных в данную версию изменений. Если заменен ссылочный документ, на который дана датированная ссылка, то рекомендуется использовать версию этого документа с указанным выше годом утверждения (принятия). Если после утверждения настоящих рекомендаций в ссылочный документ, на который дана датированная ссылка, внесено изменение, затрагивающее положение, на которое дана ссылка, то это положение рекомендуется применять без учета данного изменения. Если ссылочный документ отменен без замены, то положение, в котором дана ссылка на него, применяется в части, не затрагивающей эту ссылку.

3 Термины, определения, обозначения и сокращения

3.1 Термины и определения

В настоящих рекомендациях использованы следующие термины в соответствии со следующими определениями:

3.1.1 **трансформ** (transform): Набор правил, в соответствии с которыми происходит обработка пакета в протоколе ESP; трансформ характеризуется типом, определяющим вид обработки (например, трансформ типа `ENCR` определяет алгоритм шифрования, режим его использования, способ формирования вектора инициализации и т.д.) и уникальным идентификатором, используемым в протоколе IKEv2 при согласовании трансформов в процессе создания защищенного соединения.

3.1.2 **защищенное соединение** (Security Association; SA): Логическое объединение используемых трансформов, а также сеансовых ключей, позволяющее осуществлять защиту трафика.

3.1.3 **селектор трафика** (traffic selector): Совокупность сетевых характеристик, позволяющих выделить некоторое подмножество IP-пакетов; в общем случае является списком элементов, каждый из которых включает диапазон IP-адресов назначения и источника, сетевой протокол (или «любой» протокол) и диапазон портов назначения и источника (для протоколов TCP и UDP).

3.1.4 **инкапсуляция** (encapsulation): Упаковка сообщений протокола в сообщения другого протокола.

3.1.5 **декапсуляция** (decapsulation): Извлечение сообщений протокола из сообщений другого протокола.

3.1.6 **соль** (salt): Непредугадываемая величина, используемая при построении одноразового вектора для трансформов шифрования, описываемых в настоящих рекомендациях.

3.2 Обозначения

В настоящих рекомендациях использованы следующие обозначения:

B_s	— множество байтовых строк длины s , $s \geq 0$. Строка $b = (b_1, \dots, b_s)$ принадлежит множеству B_s , если $b_1, \dots, b_s \in \{0, \dots, 255\}$. При $s = 0$ множество B_s состоит из единственной пустой строки длиной 0;
B^*	— множество всех байтовых строк произвольной конечной длины;
$ b $	— длина байтовой строки $b \in B^*$ (если b — пустая строка, то $ b = 0$);
$b[i..j]$	— строка $b[i..j] = (b_i, b_{i+1}, \dots, b_j) \in B_{j-i+1}$, где $1 \leq i \leq j \leq s$ и $b = (b_1, \dots, b_s) \in B_s$;
$STR_s(r)$	— строка $STR_s(r) = (b_1, \dots, b_s) \in B_s$, соответствующая числу $r = 256^{s-1} \cdot b_1 + \dots + 256 \cdot b_{s-1} + b_s \leq 256^s - 1$;
$ $	— конкатенация двух байтовых строк; для двух строк $a = (a_1, \dots, a_{s_1}) \in B_{s_1}$, $b = (b_1, \dots, b_{s_2}) \in B_{s_2}$ их конкатенацией $a b$ называется строка $c = (a_1, \dots, a_{s_1}, b_1, \dots, b_{s_2}) \in B_{s_1+s_2}$;
$\&$	— операция побитовой конъюнкции (побитовое «И»);
<code>ENCR</code>	— тип трансформ, основанного на алгоритме шифрования или AEAD-алгоритме (см. 0); как правило, трансформ типа <code>ENCR</code> обеспечивает свойство конфиденциальности, однако если он основывается на AEAD-алгоритме, то обеспечивает либо конфиденциальность и имитозащиту, либо только имитозащиту (при вырожденном шифровании);
<code>INTEG</code>	— тип трансформ, основанного на алгоритме имитозащиты, обеспечивает свойство имитозащиты; использование данного типа трансформ в протоколе ESP является опциональным, а в случае, если трансформ типа <code>ENCR</code> базируется на AEAD-алгоритме, то трансформ типа <code>INTEG</code> не должен использоваться;
<code>ESN</code>	— тип трансформ, указывающего на использование расширенных номеров пакетов.

3.3 Сокращения

В настоящих рекомендациях использованы следующие сокращения:

`AAD` — дополнительные имитозащищаемые данные (Additional Authenticated Data);

`AEAD` — шифрование с имитозащитой и ассоциированными данными (Authenticated Encryption with Associated Data);

ESN — расширенный порядковый номер пакета (Extended Sequence Number);
 ESP — протокол защиты информации в IPsec (Encapsulating Security Payload);
 ICV — имитовставка (Integrity Check Value);
 IKEv2 — протокол обмена ключами в сети Интернет версии 2 (Internet Key Exchange version 2);
 IPsec — семейство протоколов защиты информации на уровне IP-пакетов (IP Security);
 IV — вектор инициализации (Initialization Vector);
 NAT — транслятор сетевых адресов (Network Address Translator);
 SA — защищенное соединение (Security Association);
 SAD — список активных защищенных соединений (Security Association Database);
 SN — порядковый номер пакета (Sequence Number);
 SPD — политика безопасности (Security Policy Database);
 SPI — идентификатор защищенного соединения (Security Parameter Index);
 TFC — конфиденциальность потока данных (Traffic Flow Confidentiality).

Примечания

1 Если это не оговаривается отдельно, в настоящих рекомендациях предполагается, что при передаче по каналу связи все числа представляются в виде строк в формате big-endian (сетевой порядок байт).

2 В настоящих рекомендациях предполагается, что все приведенные строковые константы представляются в кодировке ASCII.

3 Байтовое представление данных, соответствующих группе полей, задается конкатенацией байтовых представлений значений полей группы. Байтовое представление значения поля, являющегося вектором элементов некоторого типа, задается конкатенацией байтовых представлений элементов данного вектора в порядке их нумерации (слева направо).

4 Краткое описание протокола ESP

Семейство протоколов IPsec предназначено для обеспечения безопасной передачи IP-пакетов между узлами IP-сети. Оно включает в себя два основных протокола:

- протокол ESP, обеспечивающий непосредственную защиту IP-пакетов;
- протокол IKEv2, обеспечивающий обмен ключами, взаимную аутентификацию сторон и создание/удаление защищенных соединений; создание защищенного соединения ESP включает в себя согласование его параметров (включая используемые трансформы), выбор SPI и вычисление сеансовых ключей.

Примечание — Протокол IKEv2 используется только для создания защищенных соединений между двумя участниками (unicast).

Настоящий документ описывает работу протокола ESP и использование в нем российских криптографических алгоритмов. Конкретная комбинация свойств безопасности, которые позволяет обеспечить протокол ESP, зависит от используемых трансформов и определяется политикой безопасности абонентов на этапе создания защищенного соединения (Security Association, SA) протоколом управления ключами.

Примечание — Хотя все перечисленные выше свойства безопасности, которые позволяет обеспечить протокол ESP, являются опциональными, недопустима ситуация, когда для конкретного защищенного соединения не обеспечивается ни конфиденциальность, ни имитозащита.

Протокол ESP работает с IP-пакетами. В случае исходящей обработки данных на вход подается IP-пакет, формируется ESP-пакет с защищенными данными и на его основе формируется новый IP-пакет, в котором данные уже содержатся в защищенном виде. Порядок формирования нового IP-пакета определяется режимом протокола ESP (туннельный или транспортный, подробнее см. 4.2). В случае транспортного режима заголовок протокола ESP добавляется в IP-пакет после IP-заголовка и перед заголовком транспортного уровня (TCP, UDP и т.п.), а в случае туннельного режима — перед внутренним IP-заголовком. Для входящего IP-пакета происходит обратная операция: при наличии в нем ESP-пакета содержащиеся в ESP-пакете данные извлекаются, и восстанавливается исходный IP-пакет.

4.1 Формат ESP-пакета

ESP-пакет состоит из набора полей, содержащих полезную нагрузку (исходный пакет) в защищенном виде, а также служебную информацию для корректного восстановления исходного пакета на приемной стороне. ESP-пакет формируется из исходного IP-пакета в соответствии с параметрами конкретного защищенного соединения ESP (ESP SA).

Формат ESP-пакета приведен на рисунке 1. Заголовок протокола, непосредственно предшествующего заголовку ESP в IP-пакете (например, IPv4, IPv6 и т.п.), должен содержать значение 50 в поле, идентифицирующем следующий протокол (то есть ESP).

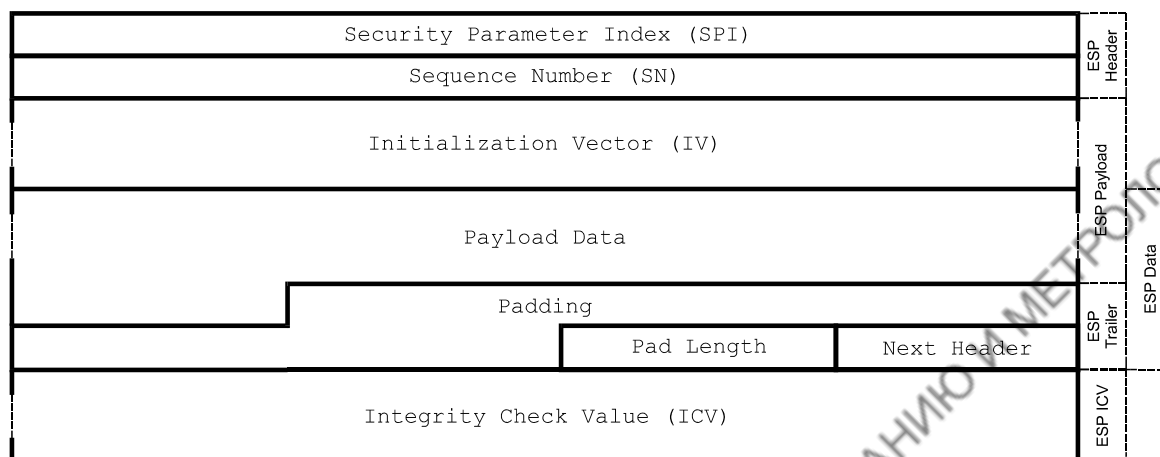


Рисунок 1 — Формат ESP-пакета

В состав ESP-пакета входят следующие поля:

Security Parameter Index	—	идентификатор защищенного соединения (4 байта), см. 4.1.1;
Sequence Number	—	порядковый номер пакета, представленный в сетевом порядке байт (4 байта), см. 4.1.2;
Initialization Vector	—	вектор инициализации (размер поля зависит от используемого трансформ типа ENCR и для установленного защищенного соединения является фиксированным);
Payload Data	—	поле данных переменной длины, может быть разбито на две части: — защищаемые данные; — TFC Padding (Traffic Flow Confidentiality Padding) — опциональное заполнение для сокрытия истинной длины передаваемых данных (см. 4.1.3);
Padding	—	заполнение (опциональное поле переменной длины: от 0 до 255 байт); должно иметь такую длину, чтобы суммарный размер полей Payload Data, Padding, Pad Length и Next Header был кратен размеру блока используемого трансформ ENCR и одновременно кратен четырем байтам;
Pad Length	—	длина поля Padding (1 байт); посылающей стороне рекомендуется выбирать минимальное значение, удовлетворяющее требованию длины Padding (см. описание поля Padding), но получатель должен принимать любое значение, которое обеспечивает надлежащее выравнивание;
Next Header	—	идентификатор протокола, данные которого содержатся в поле Payload Data (1 байт);
Integrity Check Value	—	имитовставка (размер поля зависит от используемого трансформ, обеспечивающего имитозащиту, и для установленного защищенного соединения является фиксированным).

Поля ESP-пакета (см. рисунок 1) сгруппированы в четыре группы: заголовок (ESP Header), тело (ESP Payload), завершающая часть (ESP Trailer) и имитовставка (ESP ICV). Если используемый трансформ типа ENCR обеспечивает конфиденциальность, то поля Payload Data, Padding, Pad Length и Next Header передаются в зашифрованном виде и в таком виде совокупно обозначаются как ESP Data.

Математически ESP-пакет представляется следующим образом:

$$ESP\ Packet = ESP\ Header \mid ESP\ Payload \mid ESP\ Trailer \mid ESP\ ICV. \quad (1)$$

Примечание — Далее считается, что строки *ESP Header*, ..., *ESP ICV* являются байтовыми представлениями значений групп полей *ESP Header*, ..., *ESP ICV* соответственно. Строки *SPI*, *SN*, ..., *ICV* являются байтовыми представлениями значений полей *SPI*, *SN*, ..., *ICV* соответственно.

4.1.1 Идентификатор защищенного соединения (SPI)

Идентификатор защищенного соединения SPI служит уникальным идентификатором SA на стороне получателя ESP-пакета. Поле содержит произвольное 32-битное значение. Значения SPI в диапазоне 1—255 (при представлении этого поля в виде 32-битного числа в сетевом порядке байт) зарезервированы для специальных целей и не должны использоваться в IPsec. Кроме того, значение 0 является запрещенным для SPI и не должно использоваться.

Значение SPI выбирается на этапе создания SA таким образом, чтобы получатель ESP-пакета мог однозначно определить, к какому защищенному соединению относится полученный пакет. Рекомендуется, чтобы значения SPI выбирались непредугадываемо для внешнего наблюдателя. При создании пары ESP SA протоколом IKEv2 каждый из участников выбирает SPI для того защищенного соединения, в котором он будет являться получателем пакетов, и делает это таким образом, чтобы при получении ESP-пакета он всегда мог однозначно найти ESP SA по SPI.

4.1.2 Порядковый номер пакета (SN)

Порядковый номер пакета (Sequence Number) представляет собой беззнаковое 32-битное число в сетевом порядке байт. Первый пакет, посланный по вновь созданному защищенному соединению SA, должен иметь номер 1, каждый последующий пакет — на 1 (единицу) больше. Порядковый номер пакета не должен переполняться, то есть если не согласован расширенный порядковый номер, то после отсылки $(2^{32} - 1)$ пакетов данное защищенное соединение не должно далее использоваться.

4.1.2.1 Расширенный порядковый номер пакета (ESN)

При необходимости передачи в рамках одного защищенного соединения более $(2^{32} - 1)$ пакетов может быть использован расширенный порядковый номер пакета (Extended Sequence Number). Использование расширенного номера согласовывается абонентами на этапе создания SA. Расширенный порядковый номер представляет собой 64-битное беззнаковое число. Только младшие 32 бита передаются в пакете (в поле Sequence Number), старшие 32 бита хранятся абонентами как часть состояния SA, при этом они включаются в вычисление имитовставки. При использовании расширенного порядкового номера после отсылки $(2^{64} - 1)$ пакетов данное защищенное соединение не должно далее использоваться.

4.1.2.2 Обработка порядкового номера пакета на приемной стороне

Защита от атак повторной передачи пакетов является опциональной и в некоторых случаях невыполнимой (например, если используемые в SA трансформы не обеспечивают свойства имитозащиты). Однако эта опция не согласовывается, и посылающая сторона при отсылке каждого пакета всегда должна увеличивать его порядковый номер. Приемная сторона определяет, должен ли обрабатываться порядковый номер пакета в контексте данного защищенного соединения или должен игнорироваться.

Для защиты от атак повторной передачи пакетов получатель должен проверять порядковый номер каждого входящего пакета и в случае, если такой пакет уже был получен и успешно обработан, отбрасывать вновь пришедший пакет. Эта проверка должна выполняться на самом раннем этапе обработки входящих пакетов.

В процессе передачи по сети пакеты могут перемешиваться, исчезать и множиться. Для корректной обработки порядкового номера пакета приемная сторона должна поддерживать «скользящее окно», содержащее информацию о порядковых номерах последних успешно обработанных пакетов. Минимальный размер окна — 32. Правая граница окна отражает самое большое значение порядкового номера пакета из полученных и успешно обработанных ESP-пакетов. Левая граница окна определяется таким образом: правая граница окна минус размер окна плюс один. При создании SA окно инициализируется таким образом, что левая граница устанавливается равной единице, правая — размеру окна, а окно заполняется элементами, указывающими, что соответствующий пакет еще не обработан. Пакеты с номерами меньшими, чем значение левой границы окна, отбрасываются. Пакеты с номерами, попадающими в границы окна, отбрасываются, если в соответствующем элементе окна стоит отметка, что такой пакет уже был обработан, в противном случае — обрабатываются. После успешной обработки такого пакета соответствующий элемент окна помечается как обработанный. Пакеты с номерами большими,

чем значение правой границы окна, обрабатываются. В случае если они обработаны успешно, правая граница окна устанавливается равной номеру полученного пакета, соответственно сдвигается и левая граница.

Примечание — Изменения в скользящем окне должны выполняться только после успешной обработки пакета, обязательно включающей в себя проверку его имитовставки.

4.1.2.3 Особенности обработки расширенного порядкового номера пакета на приемной стороне

При использовании расширенного порядкового номера пакета проверка на повторный пакет осуществляется аналогично тому, как это описано в 4.1.2.2, за исключением того, что все величины должны быть 64-битными. Поскольку в пакете в поле SN передаются только младшие 32 бита расширенного номера, то для получения полного 64-битного номера входящего пакета старшие 32 бита определяются эмпирически следующим образом. Обозначим:

W_{sz} — размер скользящего окна в виде 32-битного числа ($< 2^{31}$),

R — правая граница окна (64 бита),

R_l — младшие 32 бита R , то есть $R_l = R \& (2^{32} - 1)$,

R_h — старшие 32 бита R , то есть $R_h = \left(\frac{R}{2^{32}} \right) \& (2^{32} - 1)$,

L — левая граница окна (64 бита),

L_l — младшие 32 бита L , то есть $L_l = L \& (2^{32} - 1)$,

L_h — старшие 32 бита L , то есть $L_h = \left(\frac{L}{2^{32}} \right) \& (2^{32} - 1)$,

S — расширенный номер входящего пакета (64 бита),

S_l — младшие 32 бита S (из поля Sequence Number входящего пакета),

S_h — старшие 32 бита S (восстанавливаются на приемной стороне).

Возможны две ситуации расположения скользящего окна:

(1) $R_l \geq W_{sz} - 1$,

(2) $R_l < W_{sz} - 1$.

Для ситуации (1): если $S_l \geq L_l$, где $L_l = R_l - W_{sz} + 1$, то $S_h = R_h$, иначе $S_h = R_h + 1$.

Для ситуации (2): если $S_l \geq L_l$, где $L_l = R_l - W_{sz} + 1$, то $S_h = R_h - 1$, иначе $S_h = R_h$.

Данный алгоритм позволяет восстановить расширенный номер пакета S (путем комбинации S_l из пакета и полученного S_h), после чего должен использоваться алгоритм, описанный в 4.1.2.2 с поправкой на то, что все величины являются 64-битными. Приложение В содержит формализованное описание алгоритма обработки расширенного номера пакета на приемной стороне.

Примечания

1 Любые изменения в скользящем окне должны выполняться только после успешной обработки пакета с восстановленным значением S , обязательно включающей в себя проверку его имитовставки.

2 Описанный алгоритм восстановления старших 32 бита расширенного номера пакета обладает следующей особенностью: любой входящий пакет, не попадающий в окно, считается новым, а не отставшим пакетом. Иными словами, пакет, отправленный с расширенным номером S и задержавшийся в канале настолько, что в момент его получения на приемной стороне справедливо условие $R - S > W_{sz}$, в результате применения описанного алгоритма будет воспринят на приемной стороне как пакет с расширенным номером $S' = S + 2^{32}$. В результате такой пакет будет отброшен не на этапе проверки на повторный пакет, а на этапе проверки имитовставки.

3 Описанный алгоритм восстановления старших 32 бита расширенного номера пакета не будет работать в ситуации, когда происходит потеря 2^{32} и более последовательных пакетов. Хотя такое событие является маловероятным, если оно все же произойдет, то все последующие входящие пакеты данного защищенного соединения будут отбрасываться из-за неверного восстановления S_h , и, соответственно, нарушения имитозащиты. Если приемная сторона обнаружит, что в течение определенного времени все входящие пакеты с ESN были отброшены по причине нарушения имитозащиты, то она может попытаться провести ресинхронизацию следующим образом: берется произвольный отброшенный входной пакет, для него восстанавливается S_h описанным выше способом и далее инкрементируется в цикле с попыткой каждый раз проверить имитовставку пакета, используя очередное значение S_h . Количество итераций цикла должно быть ограничено. После исчерпания итераций цикла при отсутствии положительного результата можно повторить всю процедуру, используя другой отброшенный пакет. Если в какой-то момент имитовставка пакета успешно проверится, то параметры окна необходимо обновить, используя полученное значение S . Кроме того, если в процессе выполнения ресинхронизации придет пакет, который будет успешно обработан, процесс ресинхронизации следует прекратить.

При использовании расширенного номера пакета рекомендуется использовать размер окна не менее 64. Для высокоскоростных сетей или сетей с повышенными потерями рекомендуется использовать большие значения.

4.1.3 Дополнительное заполнение (TFC Padding)

В протоколе ESP может использоваться дополнительное заполнение, называемое TFC Padding (Traffic Flow Confidentiality Padding) и входящее в состав поля `Payload Data`. Оно применяется с целью сокрытия истинной длины передаваемых данных для усложнения задачи анализа сетевого трафика. Использование TFC Padding является опциональным и согласуется на этапе создания ESP SA. Опция не должна использоваться, если она не была согласована.

Технически TFC Padding осуществляется путем увеличения размеров поля `Payload Data` за счет добавления к нему на передающей стороне до шифрования произвольных данных таким образом, чтобы по результирующей длине пакета невозможно было определить длину передаваемых данных (как правило, все пакеты формируются одинаковой длины). На приемной стороне после расшифрования пакета эти данные отбрасываются. Содержимое добавляемых данных не регламентируется.

Поскольку длина TFC Padding в ESP-пакете не передается, то принимающая сторона должна определить, где заканчиваются исходные данные и начинается заполнение, которое надо отбросить. Это можно сделать, только если в самих исходных данных в явном виде присутствует их длина. В частности, TFC Padding можно использовать:

- в туннельном режиме;
- в транспортном режиме при условии, что транспортным протоколом является UDP или ICMP (или другой протокол, явно содержащий длину передаваемых данных).

Использование заполнения TFC Padding является опциональным в протоколе ESP. Другим опциональным способом достижения свойства TFC является периодическая отсылка «пустых» (dummy) пакетов, не несущих полезной информации, но, с точки зрения стороннего наблюдателя, неотличимых от других ESP-пакетов. Такие пакеты должны иметь корректные значения всех полей, кроме поля `Payload Data`, которое может содержать произвольные данные. Поле `Next Header` для таких пакетов должно иметь значение 59. На приемной стороне ESP-пакеты, которые после расшифрования содержат значение 59 в поле `Next Header`, должны отбрасываться без дальнейшей обработки.

4.2 Режимы протокола ESP

Протокол ESP может быть использован в двух режимах — туннельном и транспортном. Конкретный режим является параметром ESP SA.

4.2.1 Туннельный режим

В туннельном режиме ESP защищает весь исходный пакет, включая его заголовок. Для этого исходный (защищаемый) IP-пакет целиком помещается в поле `Payload Data` ESP-пакета вместе со своим IP-заголовком (см. рисунок 2). После этого формируется новый IP-пакет с новым IP-заголовком, включающим новые значения IP-адресов, обычно другие, чем в исходном IP-заголовке. Полезной нагрузкой нового IP-пакета становится ESP-пакет. В туннельном режиме поле `Next Header` в ESP-пакете должно иметь значение 4 для протокола IPv4 или значение 41 для протокола IPv6, указывая, что полезной нагрузкой ESP-пакета является IP-пакет. На приемной стороне после обработки ESP-пакета внешний IP-заголовок отбрасывается.



Рисунок 2 — Туннельный режим (исходный и защищенный пакеты)

В туннельном режиме допустимо применение ESP к фрагментированным IP-пакетам. Кроме того, допускается ситуация, когда версии внутреннего и внешнего IP-заголовков различаются.

4.2.2 Транспортный режим

В транспортном режиме ESP защищает только полезную нагрузку исходного IP-пакета, то есть только она помещается в поле `Payload Data` протокола ESP (см. рисунок 3). При этом в результирующем пакете используется исходный IP-заголовок с исходными адресами, но со скорректированной длиной, с указанием на ESP как на следующий протокол и с пересчитанной контрольной суммой.

В этом случае поле `Next Header` в ESP-пакете должно иметь значение, скопированное из соответствующего поля исходного IP-заголовка. В транспортном режиме протокол ESP защищает только данные исходного пакета, но не его IP-заголовок.



Рисунок 3 — Транспортный режим (исходный и защищенный пакеты)

При использовании транспортного режима ESP с протоколом IPv6 расширенные заголовки IPv6 типа «hop-by-hop», «routing» и «fragmentation» не должны защищаться протоколом ESP (то есть должны размещаться в IP-пакете до заголовка ESP). Расширенные заголовки типа «destination options» в зависимости от желаемой семантики могут как защищаться протоколом ESP (то есть помещаться внутри ESP-пакета), так и нет. В общем случае предпочтительно помещать расширенные заголовки типа «destination options» в ESP-пакет, обеспечивая тем самым их защиту.

В транспортном режиме ESP не может применяться к отдельным IP-фрагментам, поэтому в случае защиты фрагментированного IP-пакета перед применением ESP следует из IP-фрагментов восстановить исходный IP-пакет.

4.3 Дополнительное инкапсулирование ESP-пакетов

Протокол ESP определен непосредственно поверх протокола IP и выглядит как транспортный протокол (поле в IP-заголовке, идентифицирующее следующий протокол, указывает на ESP). Такая схема плохо совместима с устройствами трансляции сетевых адресов (NAT), которые в большинстве случаев полагаются на присутствие заголовка TCP или UDP после заголовка IP и при трансляции изменяют некоторые поля этих заголовков (порты). В случае с ESP это невозможно, и NAT-устройства в большинстве случаев блокируют прохождение ESP-пакетов. Чтобы обеспечить работу протокола ESP в сетях с NAT, используется инкапсулирование ESP-пакетов в UDP или в TCP.

4.3.1 Инкапсулирование ESP-пакетов в UDP

При инкапсулировании ESP-пакета в UDP заголовок UDP помещается между внешним IP-заголовком и заголовком ESP. При этом поле в IP-заголовке, идентифицирующее следующий протокол, имеет значение 17 (UDP), а порт назначения UDP заголовка должен быть равен 4500. При этом ESP-пакеты приходят на тот же порт, что и IKEv2-пакеты. Принимающая сторона может различить их, проанализировав первые 4 байта: IKEv2-пакеты, приходящие на порт 4500, всегда начинаются с четырех байт нулей, в то время как первые 4 байта ESP-заголовка содержат SPI, значение которого никогда не равно нулю. Заголовок UDP никак не защищен и отбрасывается на принимающей стороне после успешной обработки полученного пакета. На рисунке 4 показан формат ESP при инкапсулировании его в UDP.

При использовании транспортного режима совместно с UDP-инкапсуляцией и при наличии NAT на пути следования пакета в пришедшем пакете значения IP-адресов будут отличаться от исходных вследствие трансляции сетевых адресов. Поскольку поле контрольной суммы в транспортных заголовках TCP и UDP включает в себя значения сетевых адресов из IP-заголовка, то после успешной декапсуляции на приемной стороне необходимо пересчитать значение контрольной суммы транспортных заголовков таким образом, чтобы она была корректной. Исходные значения IP-адресов для пересчета контрольной суммы приемная сторона получает в момент создания SA посредством протокола IKEv2.

Инкапсулирование ESP в UDP согласовывается абонентами посредством протокола IKEv2 при создании ESP SA.

4.3.2 Инкапсулирование ESP-пакетов в TCP

Инкапсулирование ESP-пакетов в TCP может быть использовано как альтернатива инкапсулированию в UDP в тех случаях, когда протокол UDP заблокирован на промежуточных межсетевых экранах. По сравнению с инкапсулированием в UDP инкапсулирование в TCP может иметь ряд побочных

эффектов; в частности, может наблюдаться существенное снижение скорости передачи данных в тех случаях, когда защищаемым протоколом тоже является TCP. В связи с этим инкапсулирование ESP в TCP должно быть использовано только в тех случаях, когда использование UDP является невозможным. Инкапсулирование ESP в TCP согласовывается абонентами посредством протокола IKEv2 при создании ESP SA.

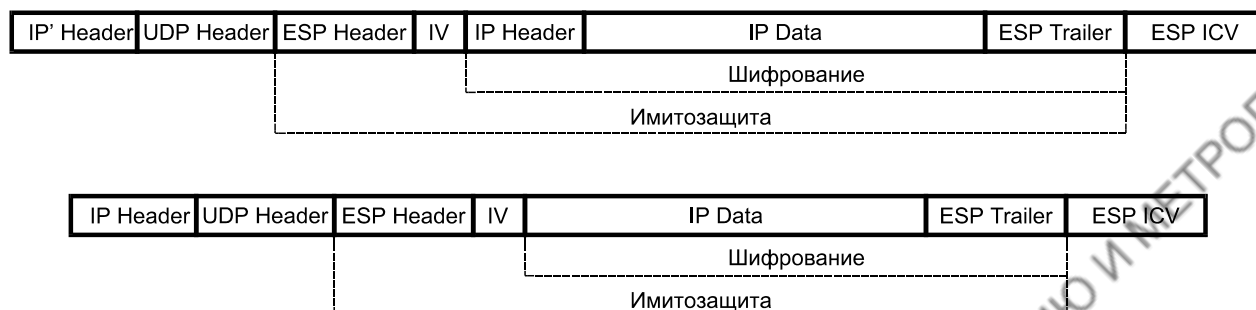


Рисунок 4 — Инкапсулирование ESP в UDP в туннельном и транспортном режимах

Инкапсулирование ESP в TCP выполняется аналогично инкапсулированию в UDP: заголовок TCP помещается между внешним IP-заголовком и заголовком ESP. При этом поле в IP-заголовке, идентифицирующее следующий протокол, имеет значение 6 (TCP), а порт назначения TCP заголовка должен быть равен 4500. Заметим, что в этом случае ESP-пакеты приходят на тот же порт, что и IKEv2-пакеты. Принимающая сторона может различить их, проанализировав первые 4 байта: IKEv2-пакеты, приходящие на порт 4500, всегда начинаются с четырех нулевых байт, в то время как первые 4 байта ESP-заголовка содержат SPI, значение которого никогда не равно нулю. Заголовок TCP никак не защищен и отбрасывается на принимающей стороне после успешной обработки полученного пакета. Для разделения отдельных ESP-пакетов и IKEv2-пакетов в потоке TCP перед каждым пакетом добавляется поле `LEN` размером в 2 байта, которое содержит в сетевом порядке байт длину следующего за ним ESP-пакета или IKEv2-пакета плюс размер самого поля (2 байта). На рисунке 5 показан формат ESP при инкапсулировании его в TCP.

Как и в случае с инкапсулированием в UDP, при использовании TCP-инкапсуляции в транспортном режиме на приемной стороне необходимо корректировать контрольную сумму в заголовках TCP и UDP, подробнее см. 5.4.

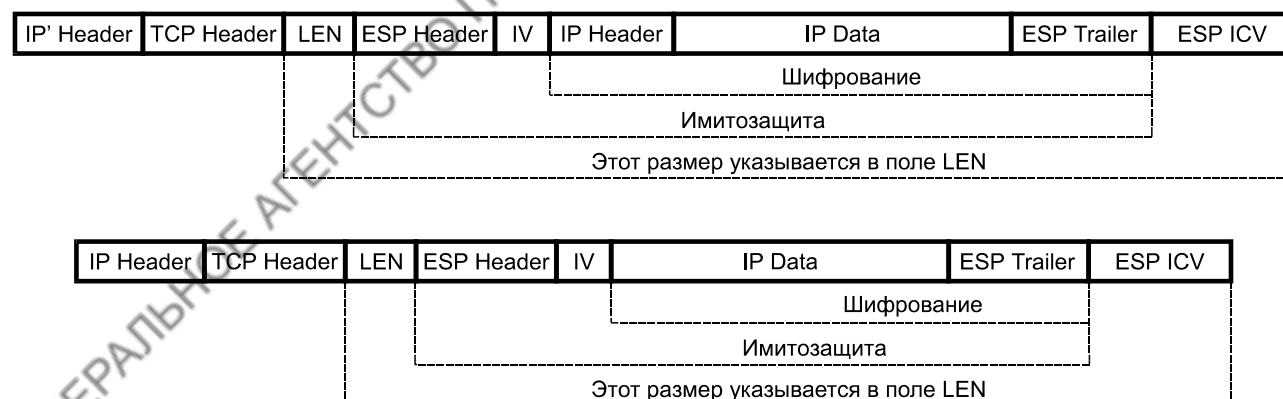


Рисунок 5 — Инкапсулирование ESP в TCP в туннельном и транспортном режимах

П р и м е ч а н и е — На рисунках 2—5 для упрощения показано, что поле IV явно включается в данные, защищаемые имитовставкой. В реальности это зависит от используемого трансформы. В частности, для AEAD-трансформов, реализующих шифрование, поле IV не включается в данные, защищаемые имитовставкой, его защита от искажения осуществляется неявным образом. Для AEAD-трансформов, реализующих только имитозащиту, поле IV явно включается в данные, защищаемые имитовставкой. См. также 6.2.8.

5 Обработка пакетов

5.1 Политика безопасности

Каждый из абонентов должен иметь политику безопасности, определяющую, какой именно трафик должен защищаться и каким образом. Политика безопасности представляет собой упорядоченный список правил, каждое из которых содержит следующую информацию:

- селектор трафика (определяет, какой сетевой трафик попадает под действие данного правила);
 - действие с трафиком, попадающим под селектор («пропустить», «отбросить», «защитить»);
 - динамический список защищенных соединений, созданных данным правилом (для действия «защитить»);
 - возможные режимы инкапсуляции (для действия «защитить»);
 - список наборов трансформов, которые могут быть использованы для защиты трафика (для действия «защитить»);
 - иные параметры, не относящиеся к безопасности (например, уровень журналирования и т.п.).
- Такой список называется Security Policy Database (SPD).

5.2 Защищенное соединение (SA)

Каждый пакет обрабатывается в некотором контексте, называемом защищенным соединением (Security Association, SA). В данном документе рассматриваются только защищенные соединения, связанные с протоколом ESP — ESP SA. Защищенные соединения создаются и удаляются динамически протоколом согласования ключей и содержат такие данные, как:

- идентификатор защищенного соединения (SPI);
- используемые трансформы:
 - трансформ шифрования или AEAD-алгоритма (типа ENCR);
 - опционально трансформ имитозащиты (типа INTEG);

П р и м е ч а н и е — В данном документе описывается использование только AEAD-алгоритмов для трансформов типа ENCR, поэтому трансформ типа INTEG не используется.

- трансформ типа ESN, определяющий использование расширенного порядкового номера пакета (принимает значение ESN, если использование ESN согласовано и noESN в противном случае);
- ключ для трансформ шифрования (K_{encr});
- опционально ключ для трансформ имитозащиты;

П р и м е ч а н и е — В данном документе описывается использование только AEAD-трансформов шифрования, поэтому ключ для трансформ имитозащиты отсутствует.

- селектор трафика (определяет, какой трафик должен защищаться данным защищенным соединением);
- порядковый номер следующего исходящего пакета (SN или ESN);
- скользящее окно номеров пакетов (на приемной стороне);
- режим инкапсуляции (туннельный/транспортный);
- туннельный IP-адрес (для туннельного режима);
- иные параметры, не относящиеся к безопасности (например, уровень журналирования и т.п.).

Протокол ESP является однонаправленным, то есть одно защищенное соединение обеспечивает защиту трафика, идущего только в одном направлении. Для защиты двунаправленного трафика (типичного в сети Интернет) абонентам необходимо создать два защищенных соединения. Протокол IKEv2 всегда создает ESP SA попарно, вне зависимости от того, будет ли защищаться однонаправленный трафик или двунаправленный.

Защищенное соединение идентифицируется значением SPI (Security Parameter Index) в заголовке ESP (см. 4.1.1).

Каждая из сторон хранит динамическую таблицу активных защищенных соединений, называемую SAD (Security Association Database).

5.3 Обработка исходящего пакета

На вход протокола ESP поступает IP-пакет. Правила SPD просматриваются от первого к последнему с целью поиска такого из них, селектор трафика которого включает сетевые параметры пакета. Если правило не найдено или содержит действие «отбросить», пакет уничтожается. Если найденное правило содержит действие «пропустить», пакет пропускается.

Если найденное правило содержит действие «защитить», то просматривается список ESP SA, созданных данным правилом, с целью поиска уже установленного защищенного соединения, селектор трафика которого включает сетевые параметры пакета. Если такового не найдено, делается запрос протоколу управления ключами для создания нового ESP SA с теми параметрами, которые заданы в правиле, а сам пакет ставится в лист ожидания. Обработка такого пакета возобновится после того, как защищенное соединение ESP будет создано.

П р и м е ч а н и е — В данном документе предполагается использование протокола управления ключами IKEv2 для создания однонаправленных (unicast) ESP SA.

Если подходящее ESP SA найдено, то пакет защищается в соответствии с указанными в нем параметрами. В частности:

а) осуществляется инкапсуляция пакета в соответствии с указанным в ESP SA режимом; при этом происходит формирование и добавление необходимых заголовков — ESP и, опционально, нового IP-заголовка;

б) данные пакета дополняются опциональным (TFC Padding) и обязательным (Padding) заполнениями, заполняются поля Pad Length и Next Header;

в) вычисляется номер (или расширенный номер) для данного пакета (путем увеличения на единицу его значения по отношению к номеру предыдущего отправленного пакета);

г) заполняются поля ESP-заголовка SPI и SN;

д) вычисляется вектор инициализации для данного пакета в соответствии с указанным в SA криптографическим преобразованием, его значение записывается в поле IV;

е) определяется ключ защиты пакета, при необходимости иницируется процесс обновления ключей (см. 6.2.10);

ж) пакет зашифровывается и имитозащита с использованием трансформации ENCR, указанного в ESP SA, ключа защиты пакета и вычисленного вектора инициализации; значение ESP ICV добавляется к пакету.

П р и м е ч а н и е — В данном документе описывается использование только AEAD-алгоритмов для трансформации типа ENCR, поэтому имитозащита данных пакета осуществляется вместе с их шифрованием;

и) опционально пакет инкапсулируется в UDP или TCP в соответствии с параметрами ESP SA;

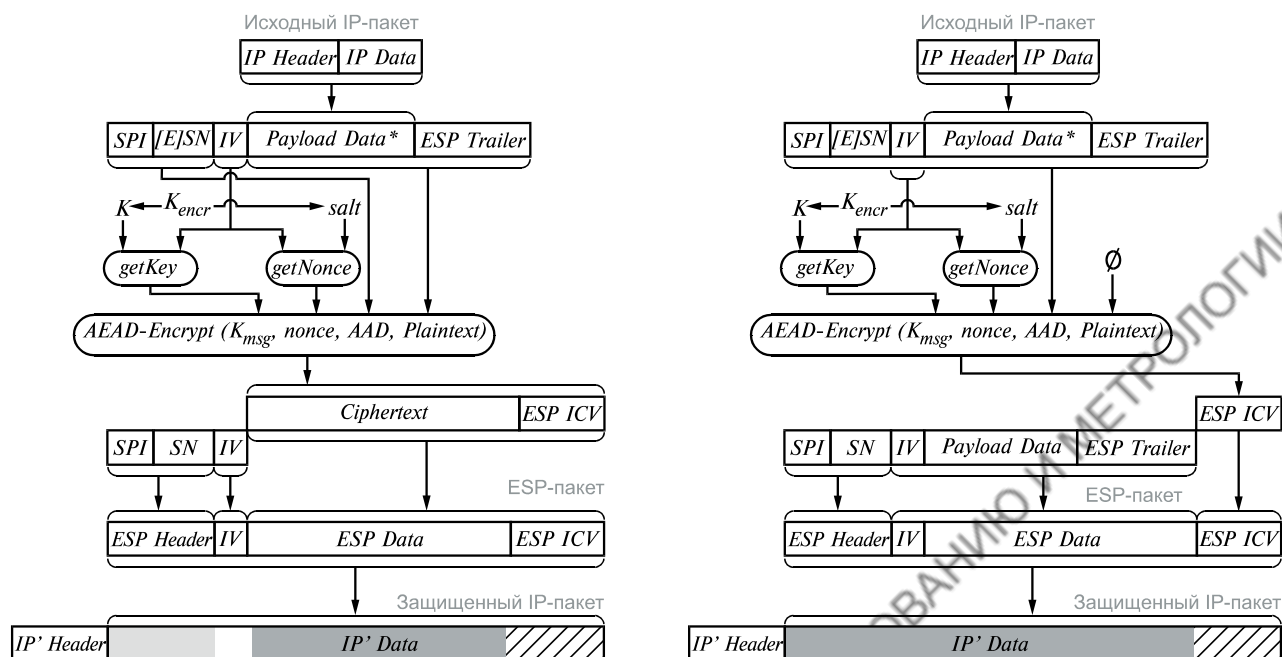
к) происходит обновление динамических полей ESP SA, таких как:

- порядковый номер (или расширенный номер) данного пакета как последнего отправленного;
- количество защищенных пакетов;
- объем защищенных данных;
- IV (при необходимости);
- прочие динамические поля SA.

П р и м е ч а н и е — Данный алгоритм предполагает, что обработка пакетов происходит в один поток. В случае многопоточной обработки пакетов в рамках одного защищенного соединения необходимо принять дополнительные меры для синхронизации проверки и изменения общих данных SA между потоками.

На рисунке 6 показана обработка исходящего пакета в случае использования туннельного режима, а на рисунке 7 — в случае использования транспортного режима.

При необходимости результирующий пакет перед отправкой может фрагментироваться (например, если его размер превысил максимальный лимит размеров пакетов для выходного сетевого интерфейса).



а) Есть шифрование

б) Нет шифрования

* — в зависимости от использования дополнительного заполнения (TFC Padding) поле **Payload Data** либо в точности равно **IP Packet**, либо представляет собой конкатенацию **IP Data** | **TFC Padding**;

■ — имитозащищенные данные;

■ — имитозащищенные и зашифрованные данные;

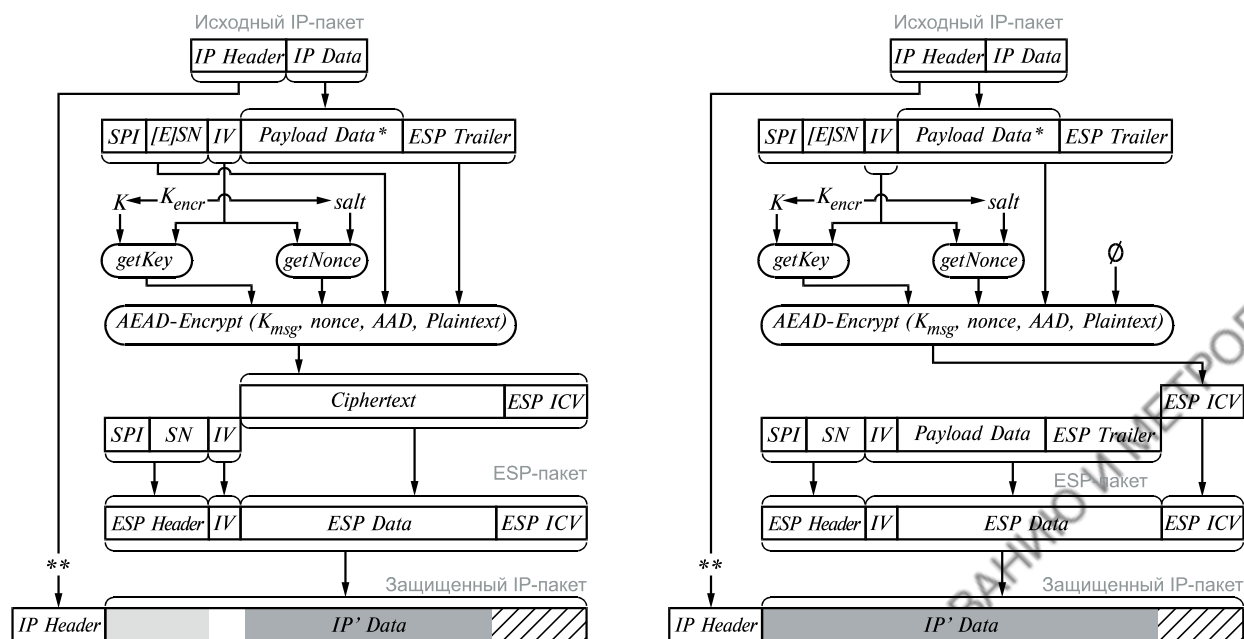
//// — значение имитовставки

Рисунок 6 — Обработка исходящего пакета в случае использования туннельного режима

Примечания

1 Функции **getKey** и **getNonce** зависят от используемого трансформатора **ENCR**.

2 Формирование AAD в общем случае зависит от используемого трансформатора **ENCR**.



а) Есть шифрование

б) Нет шифрования

- * — в зависимости от использования дополнительного заполнения (TFC Padding) поле *Payload Data* либо в точности равно *IP Data*, либо представляет собой конкатенацию *IP Data* | *TFC Padding*;
- ** — в транспортном режиме работы протокола IP-заголовок пакета остается неизменным с точностью до изменения параметра, содержащего длину пакета, с указанием на ESP как на следующий протокол и с пересчитанной контрольной суммой;

- имитозащищенные данные;
- имитозащищенные и зашифрованные данные;
- //// — значение имитовставки

Рисунок 7 — Обработка исходящего пакета в случае использования транспортного режима

5.3.1 Математическое описание процесса формирования ESP-пакета

5.3.1.1 Исходный пакет *IP Packet* представляется в виде

$$IP\ Packet = IP\ Header \mid IP\ Data, \quad (2)$$

где *IP Header* — заголовок исходного IP-пакета,
IP Data — полезная нагрузка исходного IP-пакета.

5.3.1.2 Формируется *ESP Header*.

$$ESP\ Header = SPI \mid SN, \quad (3)$$

где *SPI* — полагается равным значению *SPI* данного защищенного соединения,
SN — вычисляется путем увеличения на единицу номера предыдущего пакета, отправленного по данному защищенному соединению.

5.3.1.3 Формируется *IV*.

Если ESP SA подразумевает использование вектора инициализации, то значение *IV* вычисляется в соответствии с требованиями трансформации типа ENCR, согласованного в ходе создания ESP SA (см. подробнее 6.2.5).

5.3.1.4 Формируется *ESP Data* и *ESP ICV*.

а) Формируются поля *Payload Data* и *ESP Trailer*.

Если в ESP SA определен туннельный режим работы протокола ESP, то *Payload Data* формируется следующим образом:

$$Payload\ Data = IP\ Header \mid IP\ Data \mid TFC\ Padding, \quad (4)$$

где $TFC\ Padding \in B_s$, $s \geq 0$.

Если в ESP SA определен транспортный режим работы протокола ESP, то *Payload Data* формируется следующим образом

$$Payload\ Data = IP\ Data \mid TFC\ Padding, \quad (5)$$

где $TFC\ Padding \in B_s$, $s \geq 0$.

б) *ESP Trailer* формируется следующим образом

$$ESP\ Trailer = Padding \mid Pad\ Length \mid Next\ Header, \quad (6)$$

где *Padding* формируется следующим образом: первый байт имеет значение 1, каждый последующий — на 1 (единицу) больше. Длина заполнения выбирается таким образом, чтобы суммарная длина *Payload Data*, *Padding*, *Pad Length* и *Next Header* была кратна четырем байтам и была одновременно кратной размеру блока используемого трансформатора типа ENCR (если трансформатор накладывает такие ограничения); рекомендуется выбирать минимальный размер дополнения, удовлетворяющий этому условию;

П р и м е ч а н и е — Данный алгоритм является рекомендуемым способом заполнения поля *Padding*, однако [1] допускает использование других способов формирования дополнения; приемная сторона не должна отбрасывать пакет, если дополнение сформировано иным способом.

Pad Length содержит длину *Padding*;

Next Header в случае использования туннельного режима полагается равным 4 в случае IPv4 или 41 в случае IPv6; для транспортного режима значение копируется из поля для транспортного протокола исходного IP-заголовка; при формировании пустого (dummy) пакета *Next Header* полагается равным 59.

в) Пакет защищается с использованием параметров, указанных в ESP SA, в результате чего получаются значения *Ciphertext* и *ESP ICV*

$$Ciphertext \mid ESP\ ICV = AEAD-Encrypt(K_{msg}, nonce, AAD, Plaintext), \quad (7)$$

где алгоритм аутентифицированного шифрования *AEAD-Encrypt* задается согласованным трансформатором ENCR (см. 6.2.2).

П р и м е ч а н и е — В рамках данного документа предполагается, что в ходе выработки параметров ESP SA для трансформатора ENCR согласован AEAD-алгоритм;

значения K_{msg} и *nonce* вырабатываются из ключа K_{encr} и вектора инициализации *IV* в соответствии с алгоритмами, определенными в трансформаторе ENCR (см. 6.2.6 и 6.2.7);

значение *AAD* формируется в зависимости от согласованного трансформатора ESN и от того, согласует ли трансформатор ENCR использование шифрования (см. таблицу 5);

значение *Plaintext* формируется в зависимости от того, согласует ли трансформатор ENCR использование шифрования (см. таблицу 1).

Т а б л и ц а 1 — Формат *Plaintext*

Шифрование	<i>Plaintext</i>
Есть шифрование	<i>Payload Data</i> <i>ESP Trailer</i>
Нет шифрования	∅

г) Результирующее значение *ESP Data* формируется в зависимости от того, согласует ли трансформатор ENCR использование шифрования (см. таблицу 2).

Т а б л и ц а 2 — Формат *ESP Data*

Шифрование	<i>ESP Data</i>
Есть шифрование	<i>Ciphertext</i>
Нет шифрования	<i>Payload Data</i> <i>ESP Trailer</i>

5.3.1.5 Результирующий пакет *IP' Packet* формируется следующим образом:

$$IP'\ Packet = IP'\ Header \mid ESP\ Header \mid IV \mid ESP\ Data \mid ESP\ ICV, \quad (8)$$

где *IP' Header* — либо новый IP-заголовок с новыми значениями IP-адресов (обычно другими, чем в исходном IP-заголовке *IP Header*), если в ESP SA определен туннельный режим работы

протокола, либо исходный заголовок *IP Header* со скорректированной длиной, с указанием на ESP как на следующий протокол и с пересчитанной контрольной суммой, если в ESP SA определен транспортный режим работы протокола;
ESP Header, *IV*, *ESP Data*, *ESP ICV* — поля ESP-пакета (см. 4.1), формирование которых описано в 5.3.1.2 — 5.3.1.4.

5.4 Обработка входящего пакета

Если IP-пакет при пересылке был разбит на фрагменты, то первоначально выполняется восстановление целого IP-пакета из полученных IP-фрагментов.

При получении ESP-пакета, адресованного данному абоненту, он обрабатывается следующим образом:

а) Если пакет был инкапсулирован в UDP или в TCP, то осуществляется его декапсуляция с отбрасыванием этих заголовков.

б) Из полученного IP пакета выделяется ESP-пакет и выполняется поиск SA в SAD по значению SPI из ESP-заголовка; если защищенное соединение не найдено, пакет отбрасывается.

П р и м е ч а н и е — Данные спецификации не регламентируют конкретный механизм поиска, одним из возможных вариантов является поиск по наиболее полному соответствию триаде: IP-адрес источника, IP-адрес назначения, SPI (то есть сначала защищенное соединение ищется по всем трем критериям, если не найдено, то по двум последним, если не найдено, то только по SPI).

в) Номер полученного ESP-пакета проверяется с использованием скользящего окна из SA; если пакет идентифицирован как повторный или устаревший, он отбрасывается.

г) Определяется ключ защиты пакета, при необходимости инициируется процесс обновления ключей (см. 6.2.10).

д) Проводится проверка ICV пакета; если она прошла успешно, то выполняется расшифрование пакета с использованием трансформации *ENCR*, указанного в ESP SA, ключа защиты пакета и вектора инициализации из пакета, иначе пакет отбрасывается.

П р и м е ч а н и е — В данном документе описывается использование только AEAD-алгоритмов для трансформации типа *ENCR*, поэтому расшифрование данных происходит параллельно с проверкой ICV-пакета.

е) Отбрасываются обязательное (*Padding*) и опциональное (*TFC Padding*) дополнения.

ж) Исходный пакет реконструируется в соответствии с режимом инкапсуляции, прописанным в ESP SA, с использованием значения поля *Next Header*; если режим инкапсуляции пакета не совпадает с режимом инкапсуляции, указанным в ESP SA, пакет отбрасывается, как нарушающий политику безопасности.

и) Сетевые параметры полученного пакета проверяются на вхождение в селектор трафика ESP SA; если они не входят, то пакет отбрасывается, как нарушающий политику безопасности.

к) Происходит обновление динамических полей SA, таких как:

- границы и/или содержимое скользящего окна номеров пакетов;
- количество обработанных входящих пакетов;
- объем обработанных данных;
- прочие динамические поля SA.

л) Если полученный пакет был инкапсулирован в UDP или TCP и адрес или порт источника не совпадают с указанными в SA значениями и SA допускает обновление туннельного адреса, то туннельный адрес и порт TCP/UDP инкапсуляции обновляются значениями адреса и порта источника из пакета.

Полученный пакет отправляется в TCP/IP стек системы.

П р и м е ч а н и е — Данный алгоритм предполагает, что обработка пакетов происходит в один поток. В случае многопоточной обработки пакетов в рамках одного защищенного соединения необходимо принять дополнительные меры для синхронизации проверки и изменения общих данных SA между потоками.

5.4.1 Математическое описание процесса извлечения данных из ESP-пакета

5.4.1.1 Проводится проверка ICV полученного пакета и его расшифрование. Если проверка ICV не прошла, то пакет отбрасывается.

$$Plaintext = AEAD-Decrypt(K_{msg}, nonce, AAD, Ciphertext, ICV), \quad (9)$$

где алгоритм расшифрования и проверки имитовставки *AEAD-Decrypt* задается согласованным трансформом *ENCR* (см. 6.2.2);

значения K_{msg} и *nonce* вырабатываются из ключа K_{encr} и вектора инициализации *IV* в соответствии с алгоритмами, согласованными трансформом *ENCR* (см. 6.2.6 и 6.2.7);

значение *AAD* формируется в зависимости от согласованного трансформатора *ESN* и от того, согласует ли трансформатор *ENCR* использование шифрования (см. таблицу 5);
 значение *Ciphertext* формируется в зависимости от того, согласует ли трансформатор *ENCR* использование шифрования (см. таблицу 3).

Т а б л и ц а 3 — Формат *Ciphertext*

Шифрование	<i>Ciphertext</i>
Есть шифрование	<i>ESP Data</i>
Нет шифрования	∅

5.4.1.2 Если поле *Next Header* в *ESP Trailer* содержит значение 59, пакет отбрасывается;

5.4.1.3 От полученного значения *Plaintext* (при отсутствии шифрования — от *ESP Data*) отбрасывается *ESP Trailer*, получая значение *Payload Data*;

5.4.1.4 От значения *Payload Data* отбрасывается *TFC Padding* (при его наличии);

5.4.1.5 Полученная величина интерпретируется как *IP Data*; при этом:

а) если ESP SA использует туннельный режим, то *IP Data* интерпретируется как *IP Packet*;

б) если ESP SA использует транспортный режим, то *IP Packet* формируется следующим образом:

$$IP\ Packet = IP\ Header \mid IP\ Data, \quad (10)$$

где *IP Header* — заголовок исходного IP-пакета, в котором скорректирована длина, а поле, определяющее транспортный протокол, заменено на значение поля *Next Header*.

6 Использование российских криптографических алгоритмов в протоколе ESP

6.1 Согласование параметров ESP SA

В соответствии с настоящими рекомендациями в процессе согласования параметров ESP SA в протоколе IKEv2 для трансформатора типа *ENCR* следует использовать только параметры, представленные в 6.3. Поскольку для трансформаторов типа *ENCR* используются AEAD-алгоритмы, при согласовании параметров SA трансформатор типа *INTEG* не должен использоваться. Данные рекомендации не накладывают ограничений на согласование прочих параметров ESP SA.

6.2 Трансформы

Настоящие рекомендации определяют четыре трансформатора типа *ENCR*:

ENCR_KUZNYECHIK_MGM_KTREE

ENCR_MAGMA_MGM_KTREE

ENCR_KUZNYECHIK_MGM_MAC_KTREE

ENCR_MAGMA_MGM_MAC_KTREE

Все описанные трансформы представляют собой AEAD-алгоритмы с диверсификацией ключей. Трансформы базируются на алгоритмах «Кузнечик» (ENCR_KUZNYECHIK_MGM_KTREE и ENCR_KUZNYECHIK_MGM_MAC_KTREE) и «Магма» (ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE), описанных в ГОСТ Р 34.12—2015 (разделы 4 и 5). Алгоритмы используются в режиме MGM, описанном Р 1323565.1.026—2019.

Трансформы ENCR_KUZNYECHIK_MGM_KTREE и ENCR_MAGMA_MGM_KTREE обеспечивают конфиденциальность и имитозащиту передаваемых данных (путем аутентифицированного шифрования), трансформы ENCR_KUZNYECHIK_MGM_MAC_KTREE и ENCR_MAGMA_MGM_MAC_KTREE обеспечивают только имитозащиту (т.е. шифрование не применяется).

Примечание — Использование трансформатора типа *ENCR* с AEAD-алгоритмом для обеспечения свойства имитозащиты без конфиденциальности вместо трансформатора типа *INTEG* обусловлено тем, что в протоколе ESP только трансформатор типа *ENCR* может иметь вектор инициализации, то есть только для трансформатора типа *ENCR* можно построить дерево ключей, обеспечивающее их диверсификацию.

В протоколе IKEv2 при согласовании параметров SA трансформы ENCR_KUZNYECHIK_MGM_KTREE и ENCR_KUZNYECHIK_MGM_MAC_KTREE представлены как AEAD-алгоритмы с длиной ключа K_{encr} 352 бита, из которых первые 256 бит используются как корневой ключ SA (K), а последующие 96 бит — в качестве соли (*salt*). В свою очередь, трансформы ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE

MAC_KTREE представлены как AEAD-алгоритмы с длиной ключа 288 бит, из которых первые 256 бит используются как корневой ключ SA (K), а последующие 32 бита — в качестве соли ($salt$). Для всех трансформов длины ключей являются фиксированными, поэтому при их согласовании в IKEv2 атрибут Key Length не должен использоваться.

Данные трансформы не накладывают требований на выравнивание длины защищаемых данных (см. 4.1).

6.2.1 Блочный шифр

Определенные в настоящих рекомендациях трансформы ENCR в качестве блочного шифра используют шифры «Магма» и «Кузнечик», определенные в ГОСТ Р 34.12—2015 (см. таблицу 4). Длина блока составляет 128 бит для шифра «Кузнечик» и 64 бита для шифра «Магма», длина ключа K в обоих случаях составляет 256 бит.

Т а б л и ц а 4 — Алгоритмы блочного шифрования

Трансформ ENCR	Блочный шифр
ENCR_KUZYNECHIK_MGM_KTREE	«Кузнечик», ГОСТ Р 34.12—2015 (раздел 4)
ENCR_MAGMA_MGM_KTREE	«Магма», ГОСТ Р 34.12—2015 (раздел 5)
ENCR_KUZYNECHIK_MGM_MAC_KTREE	«Кузнечик», ГОСТ Р 34.12—2015 (раздел 4)
ENCR_MAGMA_MGM_MAC_KTREE	«Магма», ГОСТ Р 34.12—2015 (раздел 5)

6.2.2 AEAD-алгоритм

Определенные в настоящих рекомендациях трансформы ENCR в качестве AEAD-алгоритма используют блочный шифр в режиме MGM, описанном в Р 1323565.1.026—2019. Для трансформов ENCR_KUZYNECHIK_MGM_KTREE и ENCR_KUZYNECHIK_MGM_MAC_KTREE длина имитовставки S полагается равной 96 битам, для трансформов ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE используется полная имитовставка, т.е. $S = 64$.

Функция зашифрования AEAD-Encrypt задается в соответствии со следующей формулой:

$$AEAD-Encrypt(K_{msg}, nonce, AAD, Plaintext) = Ciphertext \parallel ICV, \quad (11)$$

$$\text{где} \quad (MGMnonce, AAD, Ciphertext, ICV) = MGM-Encrypt(K_{msg}, MGMnonce, AAD, Plaintext), \\ MGMnonce = nonce[1..1] \& 0x7f \parallel nonce[2..|nonce|]. \quad (12)$$

Функция расшифрования AEAD-Decrypt задается в соответствии со следующей формулой:

$$AEAD-Decrypt(K_{msg}, nonce, AAD, Ciphertext) = res, \quad (13)$$

$$\text{где} \quad res' = MGM-Decrypt(K_{msg}, MGMnonce, AAD, Ciphertext, ICV), \\ MGMnonce = nonce[1..1] \& 0x7f \parallel nonce[2..|nonce|], \quad (14)$$

$$\begin{cases} Plaintext & \text{в случае, если } res' = (AAD, Plaintext), \\ res & \text{в случае, если } res' \text{ содержит ошибку.} \end{cases}$$

6.2.3 Формат ключа трансформ

Длина ключа трансформ K_{encr} составляет 352 бита ($klen = 44$) для трансформов ENCR_KUZYNECHIK_MGM_KTREE и ENCR_KUZYNECHIK_MGM_MAC_KTREE и 288 бит ($klen = 36$) для трансформов ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE.

Настоящие рекомендации предполагают, что из ключа трансформ K_{encr} формируется корневой ключ $K \in B_{32}$ и строка $salt \in B_{klen-32}$:

$$K \parallel salt = K_{encr} \quad (15)$$

6.2.4 Алгоритм диверсификации ключа

Ключ защиты сообщения K_{msg} вычисляется на основании корневого ключа K и величин i_1 , i_2 и i_3 , передаваемых в векторе инициализации, следующим образом:

$$ESPTREE(K, i_1, i_2, i_3) = Divers(Divers(Divers(K, I_1, 0x00 \parallel i_1), I_2, STR_2(i_2)), I_3, STR_2(i_3)), \quad (16)$$

где $i_1 \in B_1$, $i_2, i_3 \in B_2$ — параметры диверсификации ключа;

I_1, I_2, I_3 — константные ASCII строки длиной 6 байт (без нулевого байта в конце строки): $I_1 = "level1"$, $I_2 = "level2"$, $I_3 = "level3"$;

$Divers(K, L, D)$ — алгоритм диверсификации ключа $K \in B_{32}$ по данным диверсификации $D \in B_2$ и метке $L \in B_6$, который задается с помощью алгоритма $KDF_GOSTR3411_2012_256$, описанного в Р 50.1.113—2016 (подраздел 4.4): $Divers(K, L, D) = KDF_{256}(K_{in}, label, seed)$.

6.2.5 Формат вектора инициализации сообщения

Каждое защищенное сообщение содержит вектор инициализации. Вектор инициализации представляет собой конкатенацию четырех величин:

$$\begin{aligned} IV &= i_1 \parallel i_2 \parallel i_3 \parallel PNUM, \\ i_2 &= STR_2(i_2), \\ i_3 &= STR_2(i_3), \\ PNUM &= STR_3(pnum), \end{aligned} \quad (17)$$

где $i_1 \in B_1$, $i_2, i_3 \in B_2$ — параметры диверсификации ключа;

$pnum \in B_3$ — порядковый номер сообщения, зашифрованного на данном ключе.

i_2 , i_3 и $PNUM$ являются представлением в сетевом порядке байт величин i_2 , i_3 и $pnum$ соответственно (см. рисунок 8). Суммарная длина вектора инициализации — 8 байт.

i_1	i_2	i_3
	$PNUM$	

Рисунок 8 — Формат вектора инициализации

Значение вектора инициализации должно быть уникальным для каждого ESP-пакета в рамках защищенного соединения. То есть для данного ключа K должно быть обеспечено различие векторов инициализации $IV = i_1 \parallel i_2 \parallel i_3 \parallel PNUM$ для всех передаваемых пакетов.

6.2.6 Ключ защиты сообщения

Ключ шифрования и имитозащиты сообщения вычисляется следующим образом:

$$K_{msg} = ESPTREE(K, i_1, i_2, i_3), \quad (18)$$

где K — корневой ключ трансформы $ENCR$;

$i_1 \in B_1$, $i_2, i_3 \in B_2$ — параметры диверсификации ключа, передаваемые в векторе инициализации (см. 6.2.5).

6.2.7 Одноразовый вектор nonce

Для защиты сообщения используется одноразовый вектор. Формат одноразового вектора зависит от используемого трансформы.

Для трансформов $ENCR_KUZYNECHIK_MGM_KTREE$ и $ENCR_KUZYNECHIK_MGM_MAC_KTREE$ одноразовый вектор *nonce* имеет размер 128 бит и формируется, как показано на рисунке 9.

<i>zero</i>	$PNUM$
<i>salt</i>	

Рисунок 9 — Формат одноразового вектора для трансформов $ENCR_KUZYNECHIK_MGM_KTREE$ и $ENCR_KUZYNECHIK_MGM_MAC_KTREE$

где *zero* — 8 бит нулей ($zero \in B_1$, $zero \equiv 0$);

$PNUM$ — порядковый номер сообщения (24 бита) из вектора инициализации (см. рисунок 8);

salt — соль (96 бит), берется из ключа трансформы K_{encr} (см. 6.2.3).

Для трансформов $ENCR_MAGMA_MGM_KTREE$ и $ENCR_MAGMA_MGM_MAC_KTREE$ одноразовый вектор *nonce* имеет размер 64 бита и формируется, как показано на рисунке 10.

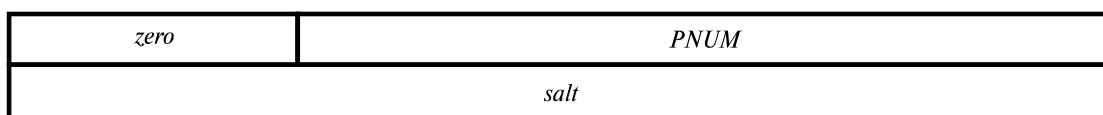


Рисунок 10 — Формат одноразового вектора для трансформов ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE

где *zero* — 8 бит нулей ($zero \in B_1$, $zero \equiv 0$);

PNUM — порядковый номер сообщения (24 бита) из вектора инициализации (см. рисунок 8);

salt — соль (32 бита), берется из ключа трансформы K_{encr} (см. 6.2.3).

Иными словами, одноразовый вектор *nonce* вычисляется на основании строки $salt \in B_{klen-32}$ и порядкового номера пакета $PNUM \in B_3$ следующим образом:

$$nonce = zero \mid PNUM \mid salt. \quad (19)$$

Для трансформов ENCR_KUZNYECHIK_MGM_KTREE и ENCR_KUZNYECHIK_MGM_MAC_KTREE одноразовый вектор *nonce* имеет размер 128 бит, для трансформов ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE одноразовый вектор *nonce* имеет размер 64 бита.

6.2.8 Формат AAD

Формат AAD (дополнительных имитозащищаемых данных) зависит от применяемого трансформы шифрования и от того, используется ли ESN (расширенный номер пакета) или нет. В случае применения трансформов ENCR_KUZNYECHIK_MGM_KTREE и ENCR_MAGMA_MGM_KTREE без использования ESN AAD формируется, как показано на рисунке 11.

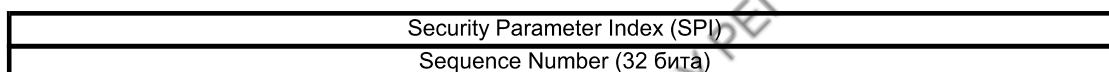


Рисунок 11 — Формат AAD для трансформов ENCR_KUZNYECHIK_MGM_KTREE и ENCR_MAGMA_MGM_KTREE без ESN

Для этих же трансформов в случае использования ESN AAD формируется, как показано на рисунке 12.

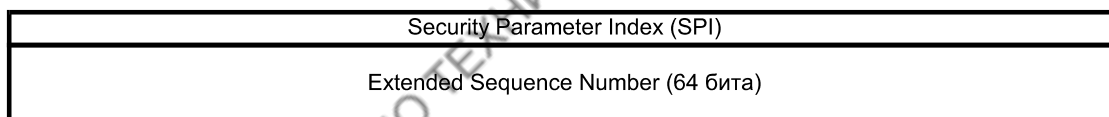


Рисунок 12 — Формат AAD для трансформов ENCR_KUZNYECHIK_MGM_KTREE и ENCR_MAGMA_MGM_KTREE с ESN

В случае применения трансформов ENCR_KUZNYECHIK_MGM_MAC_KTREE и ENCR_MAGMA_MGM_MAC_KTREE без использования ESN AAD формируется, как показано на рисунке 13.

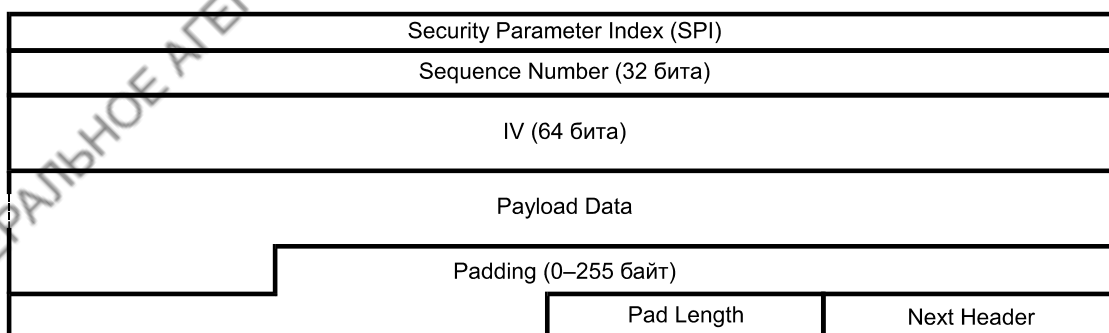


Рисунок 13 — Формат AAD для трансформов ENCR_KUZNYECHIK_MGM_MAC_KTREE и ENCR_MAGMA_MGM_MAC_KTREE без ESN

Для этих же трансформов в случае использования ESN AAD формируется, как показано на рисунке 14.

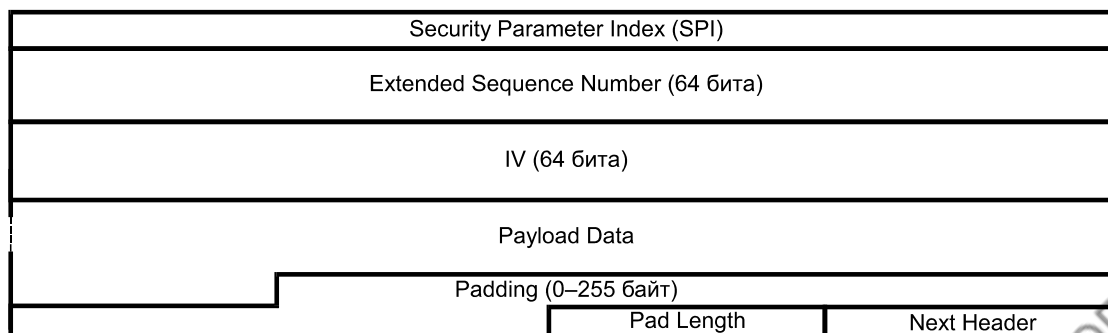


Рисунок 14 — Формат AAD для трансформов ENCR_KUZNYECHIK_MGM_MAC_KTREE и ENCR_MAGMA_MGM_MAC_KTREE с ESN

Таблица 5 описывает формирование AAD в зависимости от заданных параметров защищенного соединения.

Т а б л и ц а 5 — Формат AAD

Трансформ ENCR	Трансформ ESN	AAD
ENCR_KUZNYECHIK_MGM_KTREE ENCR_MAGMA_MGM_KTREE	noESN	<i>SPI</i> <i>SN</i>
ENCR_KUZNYECHIK_MGM_KTREE ENCR_MAGMA_MGM_KTREE	ESN	<i>SPI</i> <i>ESN</i>
ENCR_KUZNYECHIK_MGM_MAC_KTREE ENCR_MAGMA_MGM_MAC_KTREE	noESN	<i>SPI</i> <i>SN</i> <i>IV</i> <i>Payload Data</i> <i>ESP Trailer</i>
ENCR_KUZNYECHIK_MGM_MAC_KTREE ENCR_MAGMA_MGM_MAC_KTREE	ESN	<i>SPI</i> <i>ESN</i> <i>IV</i> <i>Payload Data</i> <i>ESP Trailer</i>

В случае использования расширенных номеров пакетов в AAD включаются все 64 бита номера пакета, однако поле *SN* в составе ESP-пакета включает только 32 младших бита *ESN*, а старшие 32 бита не передаются.

П р и м е ч а н и е — Заметим, что для трансформов ENCR_KUZNYECHIK_MGM_KTREE и ENCR_MAGMA_MGM_KTREE AAD не включает в себя IV, так как имитозащита IV происходит неявным образом: значение IV определяет конкретный ключ защиты сообщения и одноразовый вектор, при их несоответствии исходным значениям пакет не будет успешно проверен на приемной стороне. Однако для трансформов ENCR_KUZNYECHIK_MGM_MAC_KTREE и ENCR_MAGMA_MGM_MAC_KTREE AAD включает в себя IV, чтобы AAD представлял собой один неразрывный блок данных.

6.2.9 Выбор параметров диверсификации ключа

Выбор значений параметров диверсификации целиком определяется стороной, которая отсылает сообщение. Приемная сторона использует полученные в векторе инициализации значения параметров для вычисления ключа защиты сообщения. Каждая сторона выбирает параметры диверсификации для отсылаемых ею сообщений независимо от другой стороны.

6.2.10 Нагрузка на криптографические ключи

Предполагается, что передающая сторона ведет учет объема данных, защищенных на одном ключе K_{msg} (ключ определяется значениями i_1, i_2, i_3). Если при обработке очередного IP-пакета объем трафика будет превышен либо произойдет переполнение счетчика $pnut$, то до обработки данного пакета вырабатывается новый ключ K_{msg} путем увеличения параметра i_3 на единицу, $pnut$ при этом обнуляется. При возможном превышении нагрузки на ключ второго уровня или при переполнении i_3 параметр i_2 увеличивается на единицу, а при возможном превышении нагрузки на ключ первого уровня или при переполнении i_2 параметр i_1 увеличивается на единицу. При превышении i_1 своего максимального значения данное защищенное соединение удаляется и взамен него создается новое с новым ключом K_{encr} .

$$\begin{aligned}
 &pnim = 0, i_3 = i_3 + 1, \\
 &\text{если } i_3 > i_3^{\max}, \text{ то } i_3 = 0, i_2 = i_2 + 1, \\
 &\text{если } i_2 > i_2^{\max}, \text{ то } i_2 = 0, i_1 = i_1 + 1, \\
 &\text{если } i_1 > i_1^{\max}, \text{ то не использовать SA,} \\
 &\text{иначе } K_{msg} = ESPTREE(K, i_1, i_2, i_3),
 \end{aligned}
 \tag{20}$$

где значения $i_1^{\max}$, $i_2^{\max}$ и $i_3^{\max}$ определяются политикой безопасности, но в любом случае ограничены величинами: $i_1^{\max} \leq 255$, $i_2^{\max} \leq 65535$ и $i_3^{\max} \leq 65535$.

Передающая сторона не должна пересчитывать K_{msg} заново при отправке очередного пакета, последнее значение ключа должно сохраняться и использоваться, если значения параметров диверсификации i_1 , i_2 , i_3 не изменились по сравнению с предыдущим отправленным пакетом. Это справедливо и для промежуточных ключей в дереве.

Предполагается, что на приемной стороне также ведется учет объема данных, защищенных на одном ключе K_{msg} (ключ определяется значениями i_1 , i_2 , i_3). Если при приходе очередного ESP-пакета объемом трафика, защищенного на данном ключе K_{msg} , достигнет максимально допустимого для абонента значения нагрузки на ключ или произойдет превышение нагрузки на промежуточные ключи дерева, то обработка пакета продолжается как обычно, но при этом инициируется создание нового защищенного соединения с новым ключом K_{encr} , которое должно использоваться взамен текущего.

Приемная сторона не должна пересчитывать K_{msg} заново при получении каждого нового пакета, последнее значение ключа должно сохраняться и использоваться, если значения параметров диверсификации не изменились по сравнению с предыдущими пакетами. Рекомендуется хранить несколько последних значений, чтобы не пересчитывать ключи лишний раз при нарушении порядка следования пакетов в сети. Это справедливо и для промежуточных ключей в дереве.

Если отсылаемый IP-пакет настолько велик, что его размер превосходит максимально допустимую нагрузку на ключ, то в туннельном режиме передающая сторона должна фрагментировать его на несколько IP-фрагментов таким образом, чтобы размер любого фрагмента не превосходил допустимую нагрузку на ключ, и далее действовать по алгоритму учета накопленного объема, описанному в данном пункте. В транспортном режиме передача такого IP-пакета является невозможной.

Если на приемной стороне фиксируется получение значительного числа искаженных пакетов за короткий промежуток времени в контексте какого-то защищенного соединения, то это может служить признаком проведения атаки, особенно если при этом наблюдается хаотичное изменение значений i_1 , i_2 , i_3 и $pnim$ в получаемых пакетах. В этом случае приемная сторона может удалить данное защищенное соединение и создать идентичное ему, но с новым значением SPI и с новым ключом K_{encr} .

Примечание — В том случае, если такое поведение действительно вызвано атакой, создание нового защищенного соединения с новым значением SPI может способствовать противодействию этой атаке, так как нарушитель должен знать SPI для ее проведения. В зависимости от возможностей нарушителя это не всегда может быть тривиальной задачей.

6.3 Идентификаторы алгоритмов

Ниже приведены идентификаторы российских алгоритмов, используемых в настоящих спецификациях. Эти идентификаторы используются в протоколе IKEv2 при согласовании параметров ESP SA.

Идентификаторы для трансформации типа ENCR приведены в таблице 6.

Т а б л и ц а 6 — Идентификаторы российских алгоритмов

Трансформ ENCR	Идентификатор
ENCR_KUZNYECHIK_MGM_KTREE	32
ENCR_MAGMA_MGM_KTREE	33
ENCR_KUZNYECHIK_MGM_MAC_KTREE	34
ENCR_MAGMA_MGM_MAC_KTREE	35

Приложение А
(справочное)

Максимальный объем обрабатываемого трафика

Таблица А.1 содержит рекомендуемые значения максимально допустимого суммарного объема трафика, который обрабатывается на одном ключе K_{msg} . При достижении максимального объема должен вырабатываться новый ключ K_{msg} из дерева ключей по процедуре, описанной в 6.2.10.

Т а б л и ц а А.1 — Максимально допустимый объем трафика, обрабатываемого на одном ключе K_{msg}

Трансформ ENCR	Максимальный объем трафика
ENCR_KUZNYECHIK_MGM_KTREE ENCR_KUZNYECHIK_MGM_MAC_KTREE	2^{41} байт
ENCR_MAGMA_MGM_KTREE ENCR_MAGMA_MGM_MAC_KTREE	2^{28} байт

П р и м е ч а н и я

1 Указанные значения носят рекомендательный характер.

2 Учитывая то, что максимальное число пакетов, защищаемых на одном ключе K_{msg} , не может превысить 2^{24} , а максимальный размер IP-пакета не превышает 2^{16} байт (это ограничение действует как на защищаемый пакет, так и на IP-пакет, получающийся в результате применения к защищаемому пакету протокола ESP), суммарный объем трафика, защищаемого на одном ключе K_{msg} , не может превысить 2^{40} байт. Таким образом, ограничение 2^{41} байт для трансформов ENCR_KUZNYECHIK_MGM_KTREE и ENCR_KUZNYECHIK_MGM_MAC_KTREE является недостижимым.

При использовании трансформов, базирующихся на шифре «Магма» (ENCR_MAGMA_MGM_KTREE и ENCR_MAGMA_MGM_MAC_KTREE), рекомендуется ограничивать количество допустимых ключей K_{msg} для каждого корневого ключа K значением меньшим, чем размер дерева ключей при полном переборе i_1, i_2, i_3 (например, путем введения ограничений на максимальные значения этих счетчиков).

Приложение Б (справочное)

Тестовые векторы

Данное приложение содержит тестовые векторы. Значения тестовых векторов приведены в шестнадцатеричном виде.

Б.1 Трансформ `ENCR_KUZNYESHIK_MGM_KTREE`, пример 1

K:

```
b6 18 0c 14 5c 51 2d bd 69 d9 ce a9 2c ac 1b 5c
e1 bc fa 73 79 2d 61 af 0b 44 0d 84 b5 22 cc 38
```

i1 = 00, *i2* = 0000, *i3* = 0000, *pnum* = 000000

K_{msg}:

```
2f f1 c9 0e de 78 6e 06 1e 17 b3 74 d7 82 af 7b
d8 80 bd 52 7c 66 a2 ba dc 3e 56 9a ab 27 1d a4
```

salt [12]:

```
7b 67 e6 f2 44 f9 7f 06 78 95 2e 45
```

nonce [16]:

```
00 00 00 00 7b 67 e6 f2 44 f9 7f 06 78 95 2e 45
```

IV [8]:

```
00 00 00 00 00 00 00 00
```

AAD [8]:

```
51 46 53 6b 00 00 00 01
```

plaintext [64]:

```
45 00 00 3c 23 35 00 00 7f 01 ee cc 0a 6f 0a c5
0a 6f 0a 1d 08 00 f3 5b 02 00 58 00 61 62 63 64
65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
```

ciphertext [64]:

```
18 9d 12 88 b7 18 f9 ea be 55 4b 23 9b ee 65 96
c6 d4 ea fd 31 64 96 ef 90 1c ac 31 60 05 aa 07
62 97 b2 24 bf 6d 2b e3 5f d6 f6 7e 7b 9d eb 31
85 ff e9 17 9c a9 bf 0b db af c2 3e ae 4d a5 6f
```

ESP ICV [12]:

```
50 b0 70 a1 5a 2b d9 73 86 89 f8 ed
```

ESP packet [112]:

```
45 00 00 70 00 4d 00 00 ff 32 91 4f 0a 6f 0a c5
0a 6f 0a 1d 51 46 53 6b 00 00 00 01 00 00 00 00
00 00 00 00 18 9d 12 88 b7 18 f9 ea be 55 4b 23
9b ee 65 96 c6 d4 ea fd 31 64 96 ef 90 1c ac 31
60 05 aa 07 62 97 b2 24 bf 6d 2b e3 5f d6 f6 7e
7b 9d eb 31 85 ff e9 17 9c a9 bf 0b db af c2 3e
ae 4d a5 6f 50 b0 70 a1 5a 2b d9 73 86 89 f8 ed
```

Б.2 Трансформ `ENCR_KUZNYESHIK_MGM_KTREE`, пример 2

K:

```
b6 18 0c 14 5c 51 2d bd 69 d9 ce a9 2c ac 1b 5c
e1 bc fa 73 79 2d 61 af 0b 44 0d 84 b5 22 cc 38
```

i1 = 00, *i2* = 0001, *i3* = 0001, *pnum* = 000000

K_{msg}:

```
9a ba c6 57 78 18 0e 6f 2a f6 1f b8 d5 71 62 36
66 c2 f5 13 0d 54 e2 11 6c 7d 53 0e 6e 7d 48 bc
```

salt [12]:

```
7b 67 e6 f2 44 f9 7f 06 78 95 2e 45
```

nonce [16]:

```
00 00 00 00 7b 67 e6 f2 44 f9 7f 06 78 95 2e 45
```

IV [8]:

```
00 00 01 00 01 00 00 00
```

AAD [8]:

```
51 46 53 6b 00 00 00 10
```

```

plaintext [64]:
 45 00 00 3c 23 48 00 00 7f 01 ee b9 0a 6f 0a c5
 0a 6f 0a 1d 08 00 e4 5b 02 00 67 00 61 62 63 64
 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
 75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
ciphertext [64]:
 78 0a 2c 62 62 32 15 7b fe 01 76 32 f3 2d b4 d0
 a4 fa 61 2f 66 c2 bf 79 d5 e2 14 9b ac 1d fc 4b
 15 4b 69 03 4d c2 1d ef 20 90 6d 59 62 81 12 7c
 ff 72 56 ab f0 0b a1 22 bb 5e 6c 71 a4 d4 9a 4d
ESP ICV [12]:
 c2 2f 87 40 83 8e 3d fa ce 91 cc b8
ESP packet [112]:
 45 00 00 70 00 5c 00 00 ff 32 91 40 0a 6f 0a c5
 0a 6f 0a 1d 51 46 53 6b 00 00 00 10 00 00 01 00
 01 00 00 00 78 0a 2c 62 62 32 15 7b fe 01 76 32
 f3 2d b4 d0 a4 fa 61 2f 66 c2 bf 79 d5 e2 14 9b
 ac 1d fc 4b 15 4b 69 03 4d c2 1d ef 20 90 6d 59
 62 81 12 7c ff 72 56 ab f0 0b a1 22 bb 5e 6c 71
 a4 d4 9a 4d c2 2f 87 40 83 8e 3d fa ce 91 cc b8

```

Б.3 Трансформ ENCR_MAGMA_MGM_KTREE, пример 1

```

K:
 5b 50 bf 33 78 87 02 38 f3 ca 74 0f d1 24 ba 6c
 22 83 ef 58 9b e6 f4 6a 89 4a a3 5d 5f 06 b2 03
i1 = 00, i2 = 0000, i3 = 0000, pnum = 000000
Kmsg:
 25 65 21 e2 70 b7 4a 16 4d fc 26 e6 bf 0c ca 76
 5e 9d 41 02 7d 4b 7b 19 76 2b 1c c9 01 dc de 7f
salt [4]:
 cf 36 63 12
nonce [8]:
 00 00 00 00 cf 36 63 12
IV [8]:
 00 00 00 00 00 00 00 00
AAD [8]:
 c8 c2 b2 8d 00 00 00 01
plaintext [64]:
 45 00 00 3c 24 2d 00 00 7f 01 ed d4 0a 6f 0a c5
 0a 6f 0a 1d 08 00 de 5b 02 00 6d 00 61 62 63 64
 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
 75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
ciphertext [64]:
 fa 08 40 33 2c 4f 3f c9 64 4d 8c 2c 4a 91 7e 0c
 d8 6f 8e 61 04 03 87 64 6b b9 df bd 91 50 3f 4a
 f5 d2 42 69 49 d3 5a 22 9e 1e 0e fc 99 ac ee 9e
 32 43 e2 3b a4 d1 1e 84 5c 91 a7 19 15 52 cc e8
ESP ICV [8]:
 5f 4a fa 8b 02 94 0f 5c
ESP packet [108]:
 45 00 00 6c 00 62 00 00 ff 32 91 3e 0a 6f 0a c5
 0a 6f 0a 1d c8 c2 b2 8d 00 00 00 01 00 00 00 00
 00 00 00 00 fa 08 40 33 2c 4f 3f c9 64 4d 8c 2c
 4a 91 7e 0c d8 6f 8e 61 04 03 87 64 6b b9 df bd
 91 50 3f 4a f5 d2 42 69 49 d3 5a 22 9e 1e 0e fc
 99 ac ee 9e 32 43 e2 3b a4 d1 1e 84 5c 91 a7 19
 15 52 cc e8 5f 4a fa 8b 02 94 0f 5c

```

Б.4 Трансформ ENCR_MAGMA_MGM_KTREE, пример 2

```

K:
 5b 50 bf 33 78 87 02 38 f3 ca 74 0f d1 24 ba 6c
 22 83 ef 58 9b e6 f4 6a 89 4a a3 5d 5f 06 b2 03
i1 = 00, i2 = 0001, i3 = 0001, pnum = 000000

```

```

Kmsg:
20 e0 46 d4 09 83 9b 23 f0 66 a5 0a 7a 06 5b 4a
39 24 4f 0e 29 ef 1e 6f 2e 5d 2e 13 55 f5 da 08
salt [4]:
cf 36 63 12
nonce [8]:
00 00 00 00 cf 36 63 12
IV [8]:
00 00 01 00 01 00 00 00
AAD [8]:
c8 c2 b2 8d 00 00 00 10
plaintext [64]:
45 00 00 3c 24 40 00 00 7f 01 ed c1 0a 6f 0a c5
0a 6f 0a 1d 08 00 cf 5b 02 00 7c 00 61 62 63 64
65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
ciphertext [64]:
7a 71 48 41 a5 34 b7 58 93 6a 8e ab 26 91 40 a8
25 a7 f3 5d b9 e4 37 1f e7 6c 99 9c 9b 88 db 72
1d c7 59 f6 56 b5 b3 ea b6 b1 4d 6b d7 7a 07 1d
4b 93 78 bd 08 97 6c 33 ed 9a 01 91 bf fe a1 dd
ESP ICV [8]:
dd 5d 50 9a fd b8 09 98
ESP packet [108]:
45 00 00 6c 00 71 00 00 ff 32 91 2f 0a 6f 0a c5
0a 6f 0a 1d c8 c2 b2 8d 00 00 00 10 00 00 01 00
01 00 00 00 7a 71 48 41 a5 34 b7 58 93 6a 8e ab
26 91 40 a8 25 a7 f3 5d b9 e4 37 1f e7 6c 99 9c
9b 88 db 72 1d c7 59 f6 56 b5 b3 ea b6 b1 4d 6b
d7 7a 07 1d 4b 93 78 bd 08 97 6c 33 ed 9a 01 91
bf fe a1 dd dd 5d 50 9a fd b8 09 98

```

Б.5 Трансформ ENCR_KUZNYECHIK_MGM_MAC_KTREE, пример 1

```

K:
98 bd 34 ce 3b e1 9a 34 65 e4 87 c0 06 48 83 f4
88 cc 23 92 63 dc 32 04 91 9b 64 3f e7 57 b2 be
i1 = 00, i2 = 0000, i3 = 0000, prun = 000000
Kmsg:
98 f1 03 01 81 0a 04 1c da dd e1 bd 85 a0 8f 21
8b ac b5 7e 00 35 e2 22 c8 31 e3 e4 f0 a2 0c 8f
salt [12]:
6c 51 cb ac 93 c4 5b ea 99 62 79 1d
nonce [16]:
00 00 00 00 6c 51 cb ac 93 c4 5b ea 99 62 79 1d
IV [8]:
00 00 00 00 00 00 00 00
AAD [80]:
3d ac 92 6a 00 00 00 01 00 00 00 00 00 00 00 00
45 00 00 3c 0c f1 00 00 7f 01 05 11 0a 6f 0a c5
0a 6f 0a 1d 08 00 48 5c 02 00 03 00 61 62 63 64
65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
plaintext [0]:
ciphertext [0]:
ESP ICV [12]:
ca c5 8c e5 e8 8b 4b f3 2d 6c f0 4d
ESP packet [112]:
45 00 00 70 00 01 00 00 ff 32 91 9b 0a 6f 0a c5
0a 6f 0a 1d 3d ac 92 6a 00 00 00 01 00 00 00 00
00 00 00 00 45 00 00 3c 0c f1 00 00 7f 01 05 11
0a 6f 0a c5 0a 6f 0a 1d 08 00 48 5c 02 00 03 00
61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70
71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69
01 02 02 04 ca c5 8c e5 e8 8b 4b f3 2d 6c f0 4d

```

Б.6 Трансформ ENCR_KUZNYECHIK_MGM_MAC_KTREE, пример 2

K:
 98 bd 34 ce 3b e1 9a 34 65 e4 87 c0 06 48 83 f4
 88 cc 23 92 63 dc 32 04 91 9b 64 3f e7 57 b2 be
 i1 = 00, i2 = 0000, i3 = 0001, pnum = 000000
K_{msg}:
 02 c5 41 87 7c c6 23 f3 f1 35 91 9a 75 13 b6 f8
 a8 a1 8c b2 63 99 86 2f 50 81 4f 52 91 01 67 84
salt [12]:
 6c 51 cb ac 93 c4 5b ea 99 62 79 1d
nonce [16]:
 00 00 00 00 6c 51 cb ac 93 c4 5b ea 99 62 79 1d
IV [8]:
 00 00 00 00 01 00 00 00
AAD [80]:
 3d ac 92 6a 00 00 00 06 00 00 00 00 01 00 00 00
 45 00 00 3c 0c fb 00 00 7f 01 05 07 0a 6f 0a c5
 0a 6f 0a 1d 08 00 43 5c 02 00 08 00 61 62 63 64
 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
 75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
plaintext [0]:
ciphertext [0]:
ESP ICV [12]:
 ba bc 67 ec 72 a8 c3 1a 89 b4 0e 91
ESP packet [112]:
 45 00 00 70 00 06 00 00 ff 32 91 96 0a 6f 0a c5
 0a 6f 0a 1d 3d ac 92 6a 00 00 00 06 00 00 00 00
 01 00 00 00 45 00 00 3c 0c fb 00 00 7f 01 05 07
 0a 6f 0a c5 0a 6f 0a 1d 08 00 43 5c 02 00 08 00
 61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70
 71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69
 01 02 02 04 ba bc 67 ec 72 a8 c3 1a 89 b4 0e 91

Б.7 Трансформ ENCR_MAGMA_MGM_MAC_KTREE, пример 1

K:
 d0 65 b5 30 fa 20 b8 24 c7 57 0c 1d 86 2a e3 39
 2c 1c 07 6d fa da 69 75 74 4a 07 a8 85 7d bd 30
 i1 = 00, i2 = 0000, i3 = 0000, pnum = 000000
K_{msg}:
 4c 61 45 99 a0 a0 67 f1 94 87 24 0a e1 00 e1 b7
 ea f2 3e da f8 7e 38 73 50 86 1c 68 3b a4 04 46
salt [4]:
 88 79 8f 29
nonce [8]:
 00 00 00 00 88 79 8f 29
IV [8]:
 00 00 00 00 00 00 00 00
AAD [80]:
 3e 40 69 9c 00 00 00 01 00 00 00 00 00 00 00 00
 45 00 00 3c 0e 08 00 00 7f 01 03 fa 0a 6f 0a c5
 0a 6f 0a 1d 08 00 36 5c 02 00 15 00 61 62 63 64
 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
 75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04
plaintext [0]:
ciphertext [0]:
ESP ICV [8]:
 4d d4 25 8a 25 35 95 df
ESP packet [108]:
 45 00 00 6c 00 13 00 00 ff 32 91 8d 0a 6f 0a c5
 0a 6f 0a 1d 3e 40 69 9c 00 00 00 01 00 00 00 00
 00 00 00 00 45 00 00 3c 0e 08 00 00 7f 01 03 fa

```

0a 6f 0a c5 0a 6f 0a 1d 08 00 36 5c 02 00 15 00
61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70
71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69
01 02 02 04 4d d4 25 8a 25 35 95 df

```

Б.8 Трансформ ENCR_MAGMA_MGM_MAC_KTREE, пример 2

K:

```

d0 65 b5 30 fa 20 b8 24 c7 57 0c 1d 86 2a e3 39
2c 1c 07 6d fa da 69 75 74 4a 07 a8 85 7d bd 30

```

i1 = 00, i2 = 0000, i3 = 0001, pnum = 000000

K_{msg} :

```

b4 f3 f9 0d c4 87 fa b8 c4 af d0 eb 45 49 f2 f0
e4 36 32 b6 79 19 37 2e 1e 96 09 ea f0 b8 e2 28

```

salt [4]:

```
88 79 8f 29
```

nonce [8]:

```
00 00 00 00 88 79 8f 29
```

IV [8]:

```
00 00 00 00 01 00 00 00
```

AAD [80]:

```

3e 40 69 9c 00 00 00 06 00 00 00 00 01 00 00 00
45 00 00 3c 0e 13 00 00 7f 01 03 ef 0a 6f 0a c5
0a 6f 0a 1d 08 00 31 5c 02 00 1a 00 61 62 63 64
65 66 67 68 69 6a 6b 6c 6d 6e 6f 70 71 72 73 74
75 76 77 61 62 63 64 65 66 67 68 69 01 02 02 04

```

plaintext [0]:

ciphertext [0]:

ESP ICV [8]:

```
84 84 a9 23 30 a0 b1 96
```

ESP packet [108]:

```

45 00 00 6c 00 18 00 00 ff 32 91 88 0a 6f 0a c5
0a 6f 0a 1d 3e 40 69 9c 00 00 00 06 00 00 00 00
01 00 00 00 45 00 00 3c 0e 13 00 00 7f 01 03 ef
0a 6f 0a c5 0a 6f 0a 1d 08 00 31 5c 02 00 1a 00
61 62 63 64 65 66 67 68 69 6a 6b 6c 6d 6e 6f 70
71 72 73 74 75 76 77 61 62 63 64 65 66 67 68 69
01 02 02 04 84 84 a9 23 30 a0 b1 96

```

Приложение В
(справочное)

Алгоритм обработки расширенного номера пакета

Алгоритм обработки расширенного номера пакета на приемной стороне. Все используемые переменные представляют собой беззнаковые целые 32-битные числа, все вычисления делаются по модулю 2^{32} . Используемые переменные описаны в 4.1.2.3.

```

If (Rl >= Wsz - 1)
  If (Sl >= Rl - Wsz + 1)
    Sh = Rh
    If (Sl <= Rl)
      If (соответствующий Sl бит сброшен)
        If (имитовставка корректная)
          Установить соответствующий Sl бит
          Продолжить обработку пакета
        Else Отбросить пакет
      Else Отбросить пакет
    Else
      If (имитовставка корректная)
        Rl = Sl (сдвинуть окно)
        Установить соответствующий Sl бит
        Продолжить обработку пакета
      Else Отбросить пакет
  Else
    Sh = Rh + 1
    If (имитовставка корректная)
      Rl = Sl (сдвинуть окно)
      Rh = Rh + 1
      Установить соответствующий Sl бит
      Продолжить обработку пакета
    Else Отбросить пакет
Else
  If (Sl >= Rl - Wsz + 1)
    Sh = Rh - 1
    If (соответствующий Sl бит сброшен)
      If (имитовставка корректная)
        Установить соответствующий Sl бит
        Продолжить обработку пакета
      Else Отбросить пакет
    Else Отбросить пакет
  Else
    Sh = Rh
    If (Sl <= Rl)
      If (соответствующий Sl бит сброшен)
        If (имитовставка корректная)
          Установить соответствующий Sl бит
          Продолжить обработку пакета
        Else Отбросить пакет
      Else Отбросить пакет
    Else
      If (имитовставка корректная)
        Rl = Sl (сдвинуть окно)
        Установить соответствующий Sl бит
        Продолжить обработку пакета
      Else Отбросить пакет

```


Библиография

- [1] IETF RFC 4303 С. Кент. Защищенное вложение IP (ESP) (Kent S., «IP Encapsulating Security Payload (ESP)», RFC 4303, DOI 10.17487/RFC4303, December 2005, <<http://www.rfc-editor.org/info/rfc4303>>)

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ТЕХНИЧЕСКОМУ РЕГУЛИРОВАНИЮ И МЕТРОЛОГИИ

УДК 681.3.06:006.354

ОКС 35. 040

Ключевые слова: криптографические протоколы, режимы, шифрование, имитовставка, ключ

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ТЕХНИЧЕСКОМУ РЕГУЛИРОВАНИЮ И МЕТРОЛОГИИ

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ТЕХНИЧЕСКОМУ РЕГУЛИРОВАНИЮ И МЕТРОЛОГИИ

Редактор *Л.В. Коретникова*
Технический редактор *В.Н. Прусакова*
Корректор *Л.С. Лысенко*
Компьютерная верстка *Е.О. Асташина*

Сдано в набор 01.02.2021. Подписано в печать 09.02.2021. Формат 60×84¹/₈. Гарнитура Ариал.
Усл. печ. л. 4,18. Уч.-изд. л. 3,80.

Подготовлено на основе электронной версии, предоставленной разработчиком стандарта

Создано в единичном исполнении во ФГУП «СТАНДАРТИНФОРМ» для комплектования Федерального информационного фонда стандартов, 117418 Москва, Нахимовский пр-т, д. 31, к. 2.
www.gostinfo.ru info@gostinfo.ru

ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ТЕХНИЧЕСКОМУ РЕГУЛИРОВАНИЮ И МЕТРОЛОГИИ